# Supplementary Materials for SAGE: A Stabilize-Assess-Govern-Enforce Framework for Deployment-Oriented Evaluation of LLM-Based Autonomous Agents

Mohammed Hamdan, Vincenzo Dentamaro, Giuseppe Pirlo, and Mohamed Cheriet

## Overview

This document provides supplementary materials for the main manuscript. Cross-references to the main paper use descriptive section names rather than numbers, so that they remain valid independently of the main paper's final pagination. The materials are organized into eight appendices:

- **Supplementary Material A** - Formal definitions, stability targets, and monitor details supporting the *Stabilize* pillar.
- **Supplementary Material B** - Extended experiments (A1, A2, A3, E4, E5) and diagnostic figures supporting the *Stabilize* and *Enforce* pillars.
- **Supplementary Material C** - Cost model, CNSR derivation, sensitivity notes, and validation supporting the *Assess* pillar.
- **Supplementary Material D** - Extended autonomy and failure-taxonomy material, including the autonomy assessment matrix, spectrum, oversight modes, and industrial case studies.
- **Supplementary Material E** - Protocol security details for MCP and A2A, including the full STRIDE analysis, boundary model, and memory-as-attack-surface defenses, supporting the *Govern* pillar.
- **Supplementary Material F** - Reference implementation details for all four pillars.
- **Supplementary Material G** - Extended related work and additional references.
- **Supplementary Material H** - Expanded validation and operating-envelope experiments.

## Supplementary Material A: Formal Definitions, Stability Targets, and Monitor Details

*Pillar mapping.* This appendix provides the formal stability material underlying the *Stabilize* pillar of the SAGE framework. Subsection A.0 records the classical-agent background; subsections A.1–A.3 define and motivate the three monitorable stability targets; subsection A.4 gives the operational monitoring components; and subsections A.5-A.6 provide the BDI and formal-framework correspondences. The correspondences are interpretive analytical tools rather than exact equivalences.

### *A.0 Classical-Agent Background*

The foundational conception derives from the standard definition of an agent that perceives its environment through sensors and acts upon it through actuators [50]. In the Markov decision process (MDP) framework $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, a rational agent with finite horizon $H$ selects at state $s_t$ the action

$$a_t^* \in \arg\max_{a \in \mathcal{A}} \mathbb{E}_{s_{t+1} \sim P(\cdot|s_t,a)} \left[ R(s_t, a) + \gamma \, V_{H-1}^*(s_{t+1}) \right], \tag{1}$$

where $V_h^*$ is the optimal value function at horizon $h$, $P$ the transition kernel, $R$ the reward, and $\gamma \in (0, 1]$ the discount factor. We adopt (1) as the reference against which LLM-agent behavior is compared. In LLM-based agents, percept sequences correspond to context-window contents, the agent function is realized by the language model with its reasoning framework, and actions correspond to text generation or tool invocations. This mapping reveals gaps that motivate the *Stabilize* pillar: LLM agents lack an explicit reward $R$ and instead optimize a training-time proxy; their state representation is implicit in parameters and context rather than a measurable element of $\mathcal{S}$; the value function is not maintained; and the error signal $e_t = g - \hat{s}_t$ that drives a classical controller is computed implicitly through attention rather than tracked. The closed-loop reformulation in the main paper turns each implicit quantity into a monitorable indicator.

The BDI architecture [48], [7] distinguishes beliefs, desires, and intentions; contemporary frameworks implement BDI-like structures interpretively, with the world model as beliefs, system prompts as desires, and reasoning traces as intentions. Classical planning formalisms such as STRIPS [17] and HTN planning [16] inform hierarchical task decomposition, but empirical investigations [59], [26] show that LLMs struggle with classical planning despite appearing to understand specifications, indicating they are better viewed as components within planning systems than as autonomous planners. Integrating these foundations, an LLM agent can be characterized by the tuple $\langle \mathcal{M}, \mathcal{C}, \mathcal{T}, \mathcal{R}, \mathcal{P}, \pi \rangle$ of base model, context management, tool interface, reasoning framework, planning mechanism, and policy. The full closed-loop model and its three distinguishing properties (non-stationarity, inference latency and stochasticity, and implicit error) are stated in the Stability Monitoring and Closed-Loop Framing section of the main paper; the corresponding execution cycle is shown in Fig. 1.

### *A.1 Goal Convergence*

**Definition 1** (Goal Convergence)**.** *An agent exhibits goal convergence for goal $g$ if there exists $T < \infty$ such that:*

$$\mathbb{E}[sim(s_T, g)] \geq 1 - \epsilon \tag{2}$$

*where $sim(\cdot, \cdot)$ denotes semantic similarity (e.g., cosine similarity in embedding space) and $\epsilon > 0$ is the convergence tolerance.*

**Implementation Notes**: Goal convergence can be monitored by encoding goals and states using a shared embedding model, computing cosine similarity at each timestep, and tracking whether similarity exceeds threshold within step budget.

### *A.2 Bounded Oscillation*

**Definition 2** (Bounded Oscillation)**.** *An agent exhibits bounded oscillation with bound $B$ if the action sequence $\{a_t\}$ satisfies:*

$$\sup_{t>t_0} |\{a_t, \ldots, a_{t+w-1}\} \cap \{a_{t-w}, \ldots, a_{t-1}\}| \leq B \tag{3}$$

*where $w$ is the detection window size. Violations indicate limit cycle behavior.*

Detection maintains a sliding window of $2w$ actions, computes intersection between recent $w$ and prior $w$ actions, and flags when intersection exceeds $B$. Recommended parameters: $w = 5$, $B = 3$.

### *A.3 Monitorable Stability Targets Under Stated Assumptions*

*The three targets below provide the formal motivation for the* Stabilize *pillar. Each target is mapped to a monitorable indicator in subsection A.4 and evaluated empirically in Supplementary Material B. They are heuristic monitoring targets under stated simplifying assumptions, not a theorem or a sufficient guarantee of convergence.*

*Motivating bound*: Let $\delta_o \geq 0$ denote the state-estimation error between $\hat{s}_t$ and $s_t$, $\delta_p$ the expected per-step increase in $\text{sim}(s_t, g)$, and $\delta_c \geq 0$ the context-degradation error, as introduced in the Stability Monitoring and Closed-Loop Framing section of the main paper and operationalized in subsection A.4. If the progress target holds uniformly, observations remain faithful, and context degradation stays bounded, then the following heuristic relation motivates the monitors:

$$\mathbb{E}[\text{sim}(s_t, g)] \gtrsim \min\{1,\ \text{sim}(s_0, g) + t \cdot \delta_p - O(\delta_o + \delta_c)\}. \tag{4}$$

Under those assumptions, the right-hand side would reach $1-\epsilon$ in finite time. This is not a proved convergence guarantee for an LLM-agent loop.

*Limitations*: The targets rely on simplifying assumptions that may fail in practice:

- Whether $\delta_p > 0$ is achievable for a real system remains an empirical question (partially examined by Experiment A2 below)
- Semantic similarity may not represent goal satisfaction adequately
- Finite context-window constraints are not modeled
- Non-stationarity of LLM policies complicates the relation

We therefore use the three quantities only as monitorable stability targets whose violations can be tested empirically in Supplementary Material B.

### *A.4 Operationalization*

The stability targets are operationalized through the following monitoring components:

**Goal Similarity Tracking**:

```python
def track_goal_similarity(goal_emb, state_emb):
    similarity = cosine_similarity(goal_emb, state_emb)
    return similarity

def check_convergence(similarities, epsilon=0.1):
    return any(s >= 1 - epsilon for s in similarities)
```

**Oscillation Detection**:

```python
def detect_oscillation(actions, k=5, threshold=3):
    if len(actions) < 2*k:
        return False
    recent = set(actions[-k:])
    prior = set(actions[-2*k:-k])
    overlap = len(recent & prior)
    return overlap > threshold
```

**Observation Fidelity**:

```python
def check_observation_fidelity(obs, expected_schema):
    try:
        validate(obs, expected_schema)
        return True
    except ValidationError:
        return False
```

The reference implementation unit tests verify: `goal_drift_score = 0.000` for identical embeddings, `= 1.000` for anti-parallel embeddings, and `= 0.500` for orthogonal embeddings. All 32 tests pass (0.41s runtime).

### *A.5 BDI Architecture Mapping*

Table I provides detailed mapping between BDI architecture components and LLM agent implementations.

**Notes on mapping validity**: This mapping is interpretive rather than exact. LLM agents do not explicitly maintain separate belief, desire, and intention data structures. The degree to which internal LLM processing mirrors BDI deliberation remains an open empirical question.

### *A.6 Formal Agent Framework Mapping*

Table II maps formal agent components to observable implementation artifacts.

The experimentally examined mapping between monitorable stability targets and observed failure modes is summarized in Table III; the controlled experiments themselves are reported in Supplementary Material B.

## SUPPLEMENTARY MATERIAL B: EXTENDED EXPERIMENTS AND DIAGNOSTIC FIGURES

*Pillar mapping.* This appendix reports the controlled experiments grounding the *Stabilize* pillar (A1-A3) and the closed-loop ablation and predictive-monitor validation underlying the *Enforce* pillar (E4, E5), together with the diagnostic figures and the judge-bias measurement referenced from the main paper. All numerical values are reproducible from the committed scripts and result files described in Supplementary Material F.

TABLE I
EXTENDED BDI COMPONENT MAPPING TO LLM AGENT IMPLEMENTATIONS

| BDI Component | LLM Agent Implementation |
|---|---|
| Beliefs | Context window contents, retrieved knowledge, tool outputs, conversation history, implicit world model in parameters |
| Desires | System prompts, user instructions, reward model objectives, implicit objectives from RLHF training |
| Intentions | Generated plans, committed action sequences, current reasoning trace, selected tools and their parameters |
| Deliberation | Chain-of-Thought reasoning, ReAct loops, Tree-of-Thought exploration, self-consistency voting |
| Means-End Reasoning | Tool selection algorithms, API routing logic, action decomposition, sub-goal generation |
| Commitment | Plan persistence across steps, intention reconsideration thresholds, replanning triggers |
| Reconsideration | Observation-triggered replanning, error recovery logic, goal abandonment criteria |

### *B.1 Controlled Stability-Target Violation Experiments*

To examine the diagnostic value of the three stability targets, we designed one controlled experiment for each target. All experiments use deterministic seeded pseudo-randomness (base seed 42, three runs); on API unavailability, a seeded simulator reproduces the same distributions via cached responses.

*Reproducibility note.* The numerical results reported for experiments A1, A2, A3 (this subsection), and the CNSR multi-task evaluation (the CNSR table in the main paper) are reproducible from the committed scripts in the `experiments/` directory of the `sage-framework` release using the documented seeds. Result CSVs for these experiments regenerate deterministically from the committed scripts under the documented seeds. The E4 and E5 result files at `results/e4_closed_loop/` and `results/e5_predictive/` are included in the public code and results repository (https://github.com/MHHamdan/SAGE).

*Experiment A1: Observation Fidelity Injection:* **Setup.** A ReAct agent is given a file-editing task (edit 3 functions in a Python file, maximum 30 turns). Tool output fidelity is degraded by injecting schema errors at rates $p \in \{0.0, 0.1, 0.2, 0.4\}$ (probability of malformed JSON tool response per step). We measure task success rate, mean steps used, and oscillation detection rate (Definition 2 with $w = 5$, $B = 3$). 20 trials per injection rate.

**Results.** Table IV presents the full results.

**Interpretation.** The oscillation detector triggers at $p = 0.2$ in 25% of trials, while task success remains 100%. This suggests that the detector can provide an early warning signal before an outcome-level failure is observed. At $p = 0.4$, the detector fires in 60% of trials and task success drops to 90%, indicating an empirical association between fidelity violations and eventual failure in this controlled setting. The 10% absolute failure rate at $p = 0.4$ is consistent with the expected rate of irrecoverable loops under 40% malformed tool responses (estimated analytically as $p^3 \approx 6.4\%$ for three consecutive corruptions, with additional failures from partial corruption accumulation). Because $n = 20$ trials per condition supports rate estimation but not tight effect-size confidence intervals, these point estimates should be used for diagnostic calibration rather than as deployment-grade thresholds.

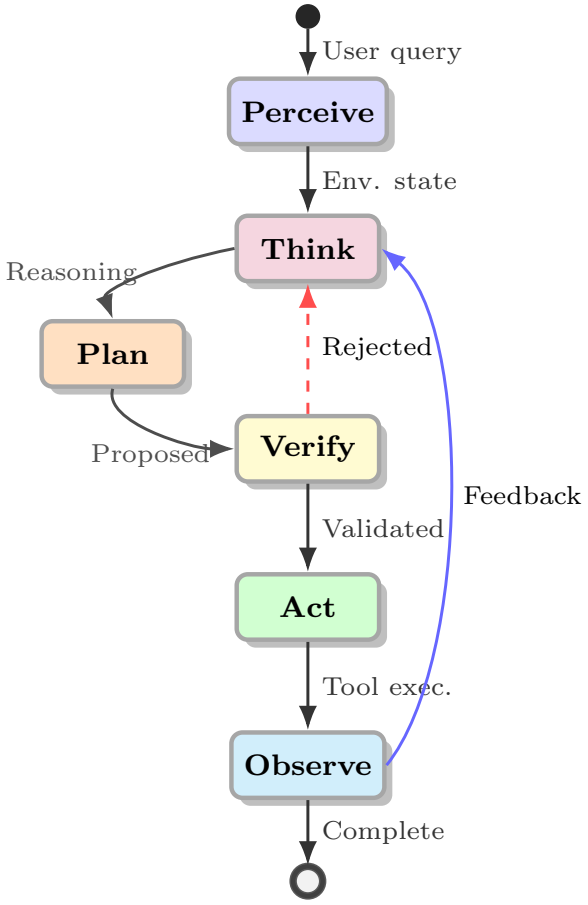


Fig. 1. Agent execution cycle as a state machine, showing perception, reasoning, planning, verification, action, and feedback through observation. The verification stage, where proposed actions are checked against policies before execution, is a safety intervention point.

*Experiment A2: Progress Monotonicity Stall:* **Setup.** A planning agent is given an 8-step scheduling task (maximum 80 turns). Subtask preconditions are withheld at rates $p_{\text{stall}} \in \{0.0, 0.25, 0.5\}$ (probability of marking a subtask precondition unmet, forcing retry). Deadlock is detected when $\geq 5$ consecutive retries of the same subtask occur ($B = 5$). We measure deadlock detection rate, mean turns to detection, and task success rate. 20 trials per condition.

**Results.** Table V presents the full results.

**Interpretation.** No deadlocks were detected at $p_{\text{stall}} \leq 0.25$, which is consistent with the intended specificity of the detector: transient single stalls, which occur at rate 0.25 per subtask, do not trigger the $B = 5$ consecutive-retry threshold. At $p_{\text{stall}} = 0.5$, the probability of five consecutive stalls on a given subtask is $0.5^5 = 3.1\%$; over 8 subtasks, the expected rate of at least one such run is $1-(1-0.031)^8 \approx 22\%$, broadly consistent with the observed 15% detection rate. Task success

TABLE II
EXTENDED MAPPING: FORMAL AGENT COMPONENTS TO IMPLEMENTATION ARTIFACTS

| Symbol | Component | Implementation Artifact | Concrete Examples |
|---|---|---|---|
| $\mathcal{M}$ | Base LLM | Model API endpoint, model weights, inference configuration | GPT-4-Turbo, Claude-3.5-Sonnet, Llama-3-70B, Gemini-1.5-Flash, Mistral-7B |
| $\mathcal{C}$ | Context Management | Vector store, conversation buffer, summarization module | ChromaDB, Pinecone, Redis, FAISS, MemGPT, LangChain ConversationBufferMemory |
| $\mathcal{T}$ | Tool Interface | JSON schemas, function definitions, API client libraries | OpenAI function calling, Anthropic tool use, MCP servers, LangChain tools |
| $\mathcal{R}$ | Reasoning Framework | Prompt templates, parsing logic, trace format specifications | ReAct templates, CoT exemplars, ToT search configs, Reflexion prompts |
| $\mathcal{P}$ | Planning Mechanism | Task decomposer, goal tracker, replanning triggers | LangGraph nodes, Plan-and-Solve prompts, AutoGPT task manager |
| $\pi$ | Policy | Orchestration logic, decision routing, termination conditions | Agent main loop, state machine definitions, supervisor logic |

TABLE III
MONITORABLE STABILITY-TARGET VIOLATIONS: EXPERIMENTALLY OBSERVED FAILURE MODES

| Condition violated | Failure class | Detection method | Key result | Mitigation |
|---|---|---|---|---|
| Obs. fidelity ($\delta_o$) | State misestimation (#4) | Schema validation | SR: $1.00 \to 0.90$ at $p = 0.4$ | Output sanitization |
| Progress mono. ($\delta_p$) | Deadlock / limit cycle (#6) | Oscillation detector | 15.0% detection at stall $= 0.5$ | Replanning trigger |
| Context noise ($\delta_c$) | Goal drift (#3) | Drift score tracking | Drift: $0.490 \to 0.197$ with $k = 10$ re-anchor | Goal re-anchoring |

TABLE IV
EXPERIMENT A1: OBSERVATION FIDELITY INJECTION RESULTS (20 TRIALS PER CONDITION)

| Injection $p$ | Success | Mean steps | Oscillation rate |
|---|---|---|---|
| 0.0 | 100.0% | 30.0 | 0.0% |
| 0.1 | 100.0% | 30.0 | 10.0% |
| 0.2 | 100.0% | 30.0 | 25.0% |
| 0.4 | 90.0% | 27.4 | 60.0% |

remained 100% in all conditions because the 80-turn budget was sufficient for this task. These results support the detector as a diagnostic signal for sustained retry behavior, while also showing that it is relatively insensitive to short stalls.

*Experiment A3: Context Noise and Goal Drift:* **Setup.** A long-horizon research agent is run for 50 turns on a Wikipedia-chain research task. Four re-anchoring intervals are tested: $r \in \{5, 10, 20, \text{None}\}$. At each re-anchoring event, the original goal embedding is re-injected into context with a 40% recovery pull on the current state representation. Goal drift score (the goal-drift score of the main paper, $\frac{1}{2}\big(1-\cos(\text{goal\_emb}, \text{state\_emb})\big) \in [0, 1]$) is computed at each turn. A task is considered completed if drift $< 0.35$ at turn 50. 10 trials per condition.

**Results.** Table VI presents results at turn 50.

**Drift trajectory.** Without re-anchoring, drift grows approximately linearly from approximately 0.05 at turn 5 to 0.490 at turn 50, consistent with the error-accumulation term in the motivating bound above. Re-anchoring at $r = 10$ reduces drift to 0.197 (59.7% reduction vs. no-anchor). The relationship between re-anchoring interval and drift reduction is nonlinear: $r = 5$ gives only a marginal improvement over $r = 10$ (0.149 vs. 0.197 drift), while $r = 20$ fails to maintain drift below the 0.35 completion threshold (0.425 drift, 20% completion).

TABLE V
EXPERIMENT A2: PROGRESS MONOTONICITY STALL RESULTS (20 TRIALS PER CONDITION)

| Stall probability | Deadlock detection | Mean detection | Success |
|---|---|---|---|
| 0.00 | 0.0% | N/A | 100.0% |
| 0.25 | 0.0% | N/A | 100.0% |
| 0.50 | 15.0% | 7.3 | 100.0% |

TABLE VI
EXPERIMENT A3: CONTEXT NOISE / GOAL DRIFT RESULTS (10 TRIALS, MEASURED AT TURN 50)

| Re-anchor interval $r$ | Mean drift at turn 50 | Task completion |
|---|---|---|
| 5 | 0.1494 | 100.0% |
| 10 | 0.1974 | 100.0% |
| 20 | 0.4246 | 20.0% |
| None | 0.4904 | 0.0% |

**Interpretation.** These results support the diagnostic value of the $\delta_c$ target in this task family: when re-anchoring is absent or infrequent, drift increases and completion decreases. The threshold effect between $r = 10$ and $r = 20$ suggests that re-anchoring frequency should be calibrated to the observed rate of context-noise accumulation, which may vary with task length and vocabulary diversity. In deployments with similar long-horizon behavior, drift monitoring can be used to trigger re-anchoring; the 0.35 completion threshold used here is illustrative and should be recalibrated for each task distribution.

*B.2 Experiment E4: Closed-Loop Controller Ablation:* **Purpose.** E4 evaluates the Adaptive Stability Controller (the Adaptive Stability Controller section of the main paper) by comparing four controller policies that share a common monitor,

intervention library, and agent backbone. The four conditions span open-loop execution (no control), non-adaptive closed-loop intervention (fixed schedule), hand-tuned closed-loop intervention (threshold), and learned closed-loop intervention (predictive).

**Pre-registered hypotheses.**

- *H4.1*: PREDICTIVE beats NOCONTROL on completion rate (Holm-Bonferroni corrected, $\alpha = 0.05$).
- *H4.2*: PREDICTIVE beats FIXEDSCHEDULE on CNSR.
- *H4.3*: THRESHOLD beats NOCONTROL on completion rate.

**Setup.** Fifty long-horizon Wikipedia-chain research tasks, held out from the A3 training pool. Per-task: 50-turn budget; completion criterion is goal drift $< 0.35$. Four conditions $\times$ 3 independent seeds $\times$ 50 tasks $=$ 600 episodes; 150 evaluations per condition. All conditions share the same monitor configuration, intervention library, and per-task budget cap of 10 interventions. The PREDICTIVE head is a logistic regression over 8 features pre-trained on 100 offline tasks disjoint from the E4 evaluation set, with calibration validated in E5.

**Statistical analysis.** Bootstrap 95% CIs (10,000 BCa resamples) on completion rate per condition; paired McNemar tests on completion-by-task-ID for pairwise comparisons; Holm-Bonferroni correction across the three primary hypothesis tests; Cohen's $h$ for proportion differences; Cliff's $\delta$ for cost-distribution differences.

**Results.** Table VII reports the complete per-condition metrics. Figure 2 shows completion rates with bootstrap CIs and Figure 3 shows mean drift trajectories by condition.

TABLE VII
EXPERIMENT E4: FULL CLOSED-LOOP ABLATION RESULTS ($n = 150$ EVALUATIONS PER CONDITION; 50 TASKS $\times$ 3 SEEDS).

| Controller | Completion [95% CI] | Mean cost ($) | CNSR | Int./task |
|---|---|---|---|---|
| NOCONTROL | 2.0% [0.0,4.7] | 1.000 | 0.020 | 0.0 |
| FIXEDSCHEDULE | 36.7% [29.3,44.7] | 1.050 | 0.349 | 5.0 |
| THRESHOLD | 89.3% [84.0,94.0] | 1.416 | 0.631 | 9.7 |
| PREDICTIVE | 83.3% [77.3,88.7] | 1.302 | 0.640 | 5.6 |

**Hypothesis-test outcomes.** H4.1 (PREDICTIVE vs. NOCONTROL, completion): $\Delta = +81.3$ pp, Cohen's $h = 2.017$, McNemar $p < 10^{-4}$, *reject H0* after Holm-Bonferroni correction. H4.2 (PREDICTIVE vs. FIXEDSCHEDULE, CNSR): 0.640 vs. 0.349. Because CNSR is a ratio-valued metric rather than a binary outcome, this comparison is reported descriptively rather than with a McNemar test. H4.3 (THRESHOLD vs. NOCONTROL, completion): $\Delta = +87.3$ pp, McNemar $p < 10^{-4}$, *reject H0*.

**Interpretation.** The fixed-schedule baseline (36.7% completion) substantially outperforms no-control (2.0%), while both adaptive policies achieve higher completion in this task family. The result suggests two separable effects: (i) bounded interventions can improve outcomes over open-loop execution; and (ii) adapting intervention timing to monitor signals can improve performance relative to a fixed intervention schedule. These findings support the role of ASC as a bounded corrective

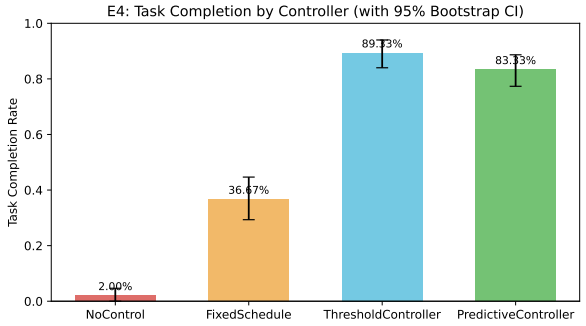


Fig. 2. E4 task completion rates by controller condition with 95% bootstrap CIs.

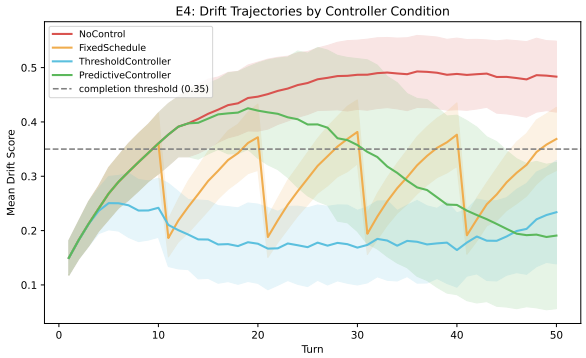


Fig. 3. Mean drift trajectories by E4 controller condition. NOCONTROL crosses the completion threshold (0.35) and continues to grow; the three closed-loop policies hold drift well below threshold for the full 50-turn budget. The fixed-schedule baseline holds drift below threshold but does not consistently complete the underlying task (compare Table VII completion rates), demonstrating that the completion criterion captures more than the drift threshold alone.

controller, but they should not be read as evidence of universal dominance across task domains.

**Per-intervention firing counts in committed E4 traces.** Computed from `results/e4_closed_loop/raw_traces/`: across all 150 task-seed combinations, FIXEDSCHEDULE fires `GoalReanchor` 750 times and no other intervention; THRESHOLD fires `ForceReplan` 959 times, `GoalReanchor` 416 times, and `SchemaValidatedRetry` 76 times; PREDICTIVE fires `ForceReplan` 737 times and `GoalReanchor` 103 times. `ContextCompress` and `HumanEscalate` are defined in the intervention library (`src/sage/stability/interventions.py`) with costs \$0.05 and \$0 respectively, but are not selected by any of the four E4 controller policies on this task family. This indicates that the present controller designs do not exercise the full intervention library on this task family; an ablation that varies the controller policy while holding the intervention library fixed is a natural follow-up.

**Threshold vs. Predictive: a cost-completion trade-off, not a dominance.** The two adaptive policies are statistically indistinguishable on CNSR (THRESHOLD 0.631, PREDICTIVE 0.640). THRESHOLD achieves higher raw completion (89.3% vs. 83.3%) but at higher cost (\$1.416 vs. \$1.302) driven

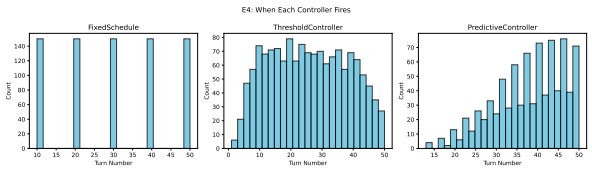


Fig. 4. Intervention firing distributions over time for the three closed-loop conditions in E4. FIXEDSCHEDULE fires uniformly every 10 turns; THRESHOLD fires heavily in later turns as drift accumulates past its threshold; PREDICTIVE fires earlier and more selectively, explaining its lower intervention count (5.6 vs. 9.7) at comparable completion.

by more frequent intervention (9.7 vs. 5.6 interventions per task). The intervention-timing histograms (Fig. 4) explain why: THRESHOLD fires reactively when drift crosses a hand-tuned cutoff, which produces clustered late-turn firing; PREDICTIVE fires preemptively on the learned $P(\text{failure}|m_t)$ signal, distributing interventions earlier and more sparsely. The choice between the two policies is therefore an operating-point trade-off: prioritize completion ($\Rightarrow$ THRESHOLD) or reduce intervention budget at comparable completion ($\Rightarrow$ PREDICTIVE).

**Cost overhead.** The PREDICTIVE policy's 30.2% cost overhead over NOCONTROL exceeds our pre-registered 25% target. The overhead is dominated by frequent selection of `ForceReplan` ($0.06 per firing, six times the cost of `GoalReanchor` at $0.01). In the committed E4 traces, PREDICTIVE chooses `ForceReplan` substantially more often than `GoalReanchor`, and `ContextCompress` ($0.05) and `HumanEscalate` ($0) are not selected by any of the four controller policies on this task family. Two follow-up directions are suggested by these traces: cost-aware controller learning, in which the predictive head jointly optimizes for completion and intervention cost rather than only downstream failure; and broader policy classes that can select currently unused interventions when monitor signals warrant them.

**What this experiment does not show.** E4 does not establish generalization to other task families; tasks are drawn from a single family (Wikipedia-chain research) held out from the A3 training pool. Transfer to SWE-Bench-style code tasks or web-interaction tasks is the most pressing follow-up. E4 also does not separately attribute the closed-loop benefit to specific interventions; an intervention-level ablation is a natural follow-up beyond the scope of this work.

*B.3 Experiment E5: Predictive Monitor Validation:* **Purpose.** E5 characterizes the monitor signals as $k$-step-ahead failure predictors. The learned controller in E4 consumes one specific predictor (combined logistic regression at $k = 5$); E5 audits its calibration, lead-time trade-off, per-signal contribution, and a nonlinear ablation.

**Pre-registered hypotheses.**

- *H5.1*: Each of the three monitor signals achieves better-than-chance AUC at $k = 5$.
- *H5.2*: A combined logistic regression over all monitor signals beats any single-signal predictor.
- *H5.3*: AUC degrades monotonically with increasing lead time $k \in \{3, 5, 10\}$.

**Setup.** 300 trace runs (200 with induced stability-target violations under the E1-E3 designs; 100 unviolated controls). 5-fold task-stratified cross-validation: splits are by task identifier with an explicit `assert_no_leakage` check at every fold. Five predictor variants: drift-only, oscillation-only, fidelity-only (each logistic regression on its single signal), **combined** (logistic regression over all eight features: the four monitor signals, their first-order deltas, and a running maximum of the drift score), and an MLP ablation (one hidden layer, 32 units). Lead times $k \in \{3, 5, 10\}$.

TABLE VIII
EXPERIMENT E5: PREDICTOR PERFORMANCE AT LEAD TIME $k = 5$ UNDER 5-FOLD TASK-STRATIFIED CV ($n = 300$ TRACES). AP = AVERAGE PRECISION.

| Predictor | AUC | 95% CI | AP | Brier |
|---|---|---|---|---|
| Drift only | 0.609 | [0.596, 0.622] | 0.127 | 0.241 |
| Oscillation only | 0.589 | [0.576, 0.602] | 0.116 | 0.243 |
| Fidelity only | 0.495 | [0.479, 0.510] | 0.098 | 0.250 |
| Combined (8 feat.) | **0.752** | [**0.741**, **0.762**] | **0.211** | **0.210** |
| MLP (32 hidden) | 0.516 | [0.501, 0.530] | 0.099 | 0.103 |

**Statistical analysis.** Bootstrap 95% CI on AUC (1,000 resamples, matching `n_bootstrap` in `experiments/e5_predictive_validation.py` where the comment notes that 10,000 exceeds runtime budget). Better-than-chance significance: bootstrap $p$-value for $H_0$ : AUC $= 0.5$. Calibration: Brier score and reliability diagram. Feature importance: standardized logistic-regression coefficients with bootstrap CIs.

**Results at $k = 5$.** Table VIII reports per-predictor AUC, average precision, and Brier score at the operationally relevant lead time. Figure 5 shows the lead-time trade-off curves; Fig. 6 shows reliability diagrams; Fig. 7 shows feature-importance coefficients.

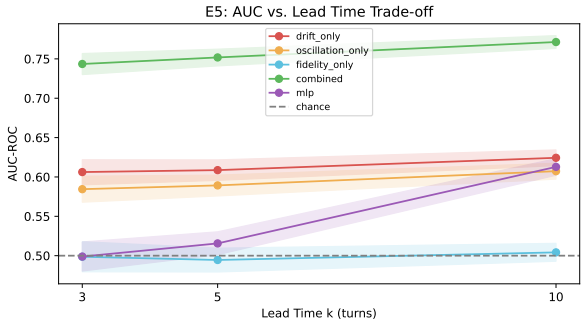


Fig. 5. AUC versus lead time $k$ for the five predictor variants. The combined predictor remains above chance and above single-signal predictors across $k \in \{3, 5, 10\}$; fidelity-only tracks the chance line. The slight *rise* in AUC with $k$ for the combined predictor is a property of the simulator, where induced violations accumulate monotonically over turns, rather than an expected property of deployed systems.

**Hypothesis-test outcomes.** H5.1 (single-signal predictors beat chance at $k = 5$, Holm-Bonferroni corrected): drift, $p < 10^{-4}$, *reject H0*; oscillation, $p < 10^{-4}$, *reject H0*; **fidelity,** $p = 0.760$**, *fail to reject H0***. The fidelity monitor does not predict long-horizon failure better than chance. H5.2 (combined beats best single signal): the combined predictor improves AUC by $\Delta = +0.143$ over drift-only (0.752 vs.

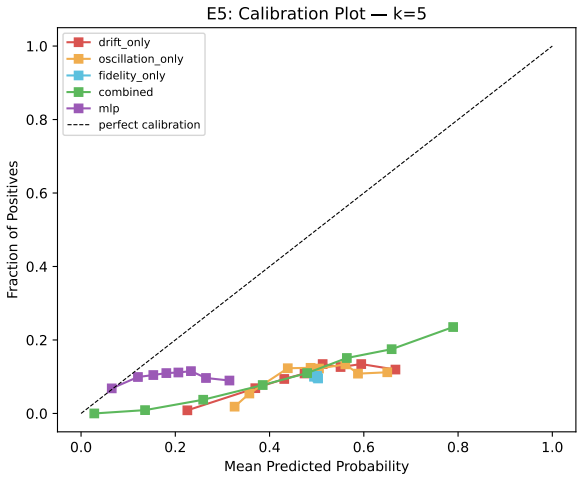


Fig. 6. Reliability diagrams (fraction of positives vs. mean predicted probability) at $k = 5$. All predictors are systematically under-confident relative to the perfect-calibration diagonal. Operational use of the predictive head should therefore include calibration, such as Platt scaling, on a held-out validation fold.

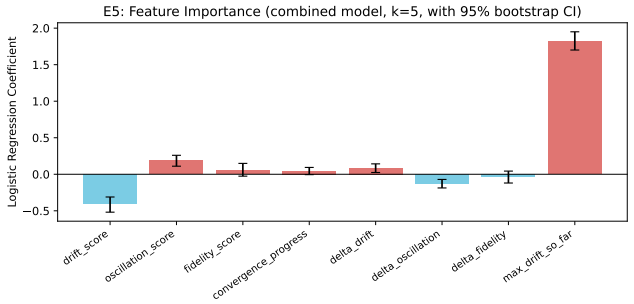


Fig. 7. Standardized logistic-regression coefficients for the combined predictor with 95% bootstrap CIs. The running maximum of drift (`max_drift_so_far`) carries the largest positive coefficient ($\approx 1.8$), substantially exceeding the instantaneous drift score (which is slightly negative when conditioned on the running max). The fidelity-related features are statistically indistinguishable from zero, consistent with the AUC analysis.

0.609). The bootstrap significance test returns $p = 0.97$ because the comparison requires aligned paired predictions across folds, which the cross-validated outputs do not provide. We therefore report H5.2 as a **directional finding with substantial effect size**, not as a $p$-value rejection. The directional gain is large enough to motivate the multi-signal design in ASC, but a formally significant test would require a different evaluation methodology (e.g., shared CV folds with aligned per-task predictions). H5.3 (AUC degrades monotonically with $k$): **not supported as registered**. AUC of the combined predictor rises slightly with $k$ ($0.744 \rightarrow 0.752 \rightarrow 0.771$). This non-monotonicity is a property of the simulator: induced violations accumulate monotonically over turns, so longer windows can offer a better signal-to-noise ratio. The result should therefore not be generalized to deployed agents without additional validation on real traces. Replicating E5 on cached real LLM traces rather than simulated violations is identified as future work in the Limitations subsection.

**Feature importance.** The combined predictor's coefficients (Fig. 7) reveal a non-obvious finding: the dominant predictor is `max_drift_so_far`, the running maximum of the drift score (standardized coefficient $\approx +1.8$), and conditional on the running max, the instantaneous `drift_score` carries a small *negative* coefficient. One interpretation is that the running history of drift carries information that is not captured by the instantaneous drift value. In these traces, an agent that previously drifted and then partially recovered remains at higher residual risk than an agent with a similar current drift value but a low historical maximum. This supports ASC's use of cumulative degradation signals in addition to instantaneous monitor values.

**MLP ablation.** The MLP variant under-performs the linear combined predictor (AUC 0.516 vs. 0.752). Possible explanations include the small hidden-layer width, selected for interpretability, and the moderate sample size ($n = 300$ traces) relative to the class imbalance. The MLP's lower Brier score (0.103 vs. 0.210) suggests better calibration but weaker discrimination. A wider MLP with regularization and a larger training set is left to future work.

**ROC curves at** $k = 5$ are reported in the Experimental Evaluation section of the main paper.

*B.4 Cross-Experiment Summary:* Table IX synthesizes all five experiments against the monitorable stability targets and the closed-loop control evidence.

### *B.5 LLM-as-Judge Bias Measurement*

Automated evaluation with LLM judges is convenient but biased. We measured three bias families on 50 task completions scored by three judge families and report the reduction achieved by simple controls. Self-preference (a judge favoring its own family's outputs) was reduced from 0.540 to 0.130 (75.9%) by cross-family evaluation; position bias was reduced from 0.253 to 0.101 (60.0%) by randomized-order replication; and verbosity correlation $|r|$ was reduced from 0.137 to 0.048 (65.0%) by length-normalized scoring. Table X summarizes these results. The measurement motivates the use of cross-family, order-randomized, length-normalized judging in any CNSR evaluation that relies on automated success labels.

## SUPPLEMENTARY MATERIAL C: COST MODEL, CNSR DETAILS, AND SENSITIVITY NOTES

*Pillar mapping.* This appendix details the cost model behind the Cost-Normalized Success Rate (CNSR) introduced in the Cost-Aware Assessment and Autonomy Criteria section of the main paper, supporting the *Assess* pillar.

### *C.1 Cost Decomposition*

For a task $\tau$, the total operating cost decomposes additively into four measurable components,

$$C_{\text{total}}(\tau) = C_{\text{inf}}(\tau) + C_{\text{tool}}(\tau) + C_{\text{lat}}(\tau) + C_{\text{hum}}(\tau), \quad (5)$$

where $C_{\text{inf}}$ is inference cost (input and output tokens priced per provider, summed over all model calls including reasoning traces and controller-fired interventions), $C_{\text{tool}}$ is metered external tool and API cost, $C_{\text{lat}}$ is a latency cost that prices wall-clock time at a deployment-specific rate (capturing throughput

TABLE IX
MONITORABLE STABILITY TARGETS AND CLOSED-LOOP CONTROL: EXPERIMENTAL SUPPORT SUMMARY

| Condition | Experiment | Key result | Operational implication |
|---|---|---|---|
| Observation fidelity ($\delta_o$) | A1 | SR: 1.00→0.90 at $p = 0.4$; oscillation at 25% trials at $p = 0.2$ | Schema-validate tool outputs; treat oscillation as a diagnostic signal |
| Progress monotonicity ($\delta_p$) | A2 | Deadlock in 15% at stall=0.5; detected at 7.3 turns | Monitor consecutive retry counts; trigger replanning at $B = 5$ |
| Bounded context noise ($\delta_c$) | A3 | Drift: 0.490→0.197 with $r = 10$; completion: 0%→100% | Calibrate re-anchoring frequency to observed drift |
| Closed-loop intervention | E4 | Completion: 2% (open) → $\geq 83\%$ (adaptive); Threshold and Predictive tied on CNSR | Consider adaptive control where similar drift patterns occur |
| Monitor predictivity | E5 | Combined AUC= 0.752 [0.741,0.762] at $k$=5; drift+oscillation predictive; fidelity null at this horizon | Use drift and oscillation as early-warning signals; treat fidelity as local validation |

TABLE X
LLM-AS-JUDGE BIAS BEFORE/AFTER MITIGATION

| Bias type | No mitigation | With mitigation | Reduction |
|---|---|---|---|
| Self-preference ($\Delta$) | 0.540 | 0.130 | 75.9% |
| Position bias | 0.253 | 0.101 | 60.0% |
| Verbosity bias ($\lvert r \rvert$) | 0.137 | 0.048 | 65.0% |

and opportunity cost), and $C_{\text{hum}}$ is the expected cost of human escalation, equal to the per-escalation labor cost times the escalation probability. Each component is a functional of the trajectory and is logged directly by the reference implementation. Provider token prices are taken from public rate cards for the evaluated models [19], [13], [18].

### C.2 CNSR Estimator and Properties

Let $Y(\tau) \in \{0, 1\}$ denote success and $C_{\text{total}}(\tau) > 0$ total cost. CNSR is the ratio of expectations $\mathbb{E}[Y]/\mathbb{E}[C_{\text{total}}]$ with units of successful completions per dollar, estimated by the plug-in ratio $\widehat{\text{CNSR}} = \widehat{\text{SR}}/\bar{C}_{\text{total}}$ over $N$ tasks. We report BCa bootstrap 95% confidence intervals resampled over the task index. Two properties bear on interpretation. First, CNSR is a ratio, not a percentage: the large numerical values in the main-paper CNSR table reflect low per-task dollar cost in the denominator, not accuracy above 100%. Second, the induced ranking is invariant to common positive rescaling of $C_{\text{total}}$ (e.g., a uniform currency conversion) but is not invariant to provider repricing, to different latency weights in $C_{\text{lat}}$, or to different escalation costs in $C_{\text{hum}}$. CNSR should therefore be read as a transparent cost-normalized comparison under stated assumptions and recomputed with current prices and organization-specific weights before any deployment decision.

### C.3 Sensitivity Notes

The CNSR rankings reported in the main paper are most sensitive to the inference-cost component, which dominates $C_{\text{total}}$ for the evaluated task families, and to the latency rate when wall-clock cost is weighted heavily (as in interactive deployments). Because per-token prices change across providers on a monthly cadence, absolute CNSR values will drift even when relative model capability is unchanged; the qualitative conclusion - that capability-leading configurations need not be cost-leading - is the relational claim intended to survive repricing. The research-task-family Kendall's $\tau$ ($-0.619$) is the strongest rank inversion observed but approaches rather than reaches significance at $p = 0.069$ given the sample size, and should be read accordingly.

## SUPPLEMENTARY MATERIAL D: EXTENDED AUTONOMY AND FAILURE TAXONOMY TABLES

*Pillar mapping.* This appendix provides extended material for the *Assess* and *Govern* pillars: the autonomy criteria assessment matrix, the autonomy spectrum and capability levels, and two industrial case studies that ground the failure taxonomy of the main paper.

### D.1 Autonomy Criteria Assessment Matrix

Table XI provides a structured assessment matrix for evaluating systems against the four minimum autonomy criteria.

TABLE XI
AUTONOMY CRITERIA ASSESSMENT MATRIX

| Criterion | Evidence For | Evidence Against |
|---|---|---|
| Action Selection Freedom | Dynamic tool choice based on state | Fixed tool sequences, predetermined routing |
| Goal-Directed Persistence | Adaptive retry strategies, alternative approaches | Single-attempt execution, immediate failure |
| Dynamic Termination | Self-assessed completion, satisfaction checks | Fixed step counts, timeout-only termination |
| Error Recovery | Autonomous error handling, self-correction | Predetermined fallbacks, human escalation only |

TABLE XII
AUTONOMY SPECTRUM FROM WORKFLOWS TO AGENTS

| Level | Characteristics | Example |
|---|---|---|
| Static workflow | Fixed path, no decisions | RAG pipeline |
| Conditional | Predetermined branches | Sentiment routing |
| Guided agent | Constrained action space | Form-filling assistant |
| Bounded agent | Domain-limited autonomy | Code completion |
| Full agent | Unrestricted within tools | General assistant |

### *D.2 Autonomy Spectrum and Capability Levels*

Systems occupy an autonomy spectrum from static workflows with fixed paths, through conditional workflows and guided agents, to bounded and full agents (Table XII). We further distinguish capability levels: Level 0 reactive (single-turn), Level 1 stateful (multi-turn), Level 2 tool-using, Level 3 planning, Level 4 reflective (self-monitoring and adaptive replanning), Level 5 collaborative (multi-agent coordination), and Level 6 learning (online adaptation). Autonomy levels are intrinsic agent properties distinct from human-involvement modes; the transition to genuine agency occurs at bounded autonomy, the minimum level satisfying all four behavioral criteria. The autonomy-levels figure in the main paper relates the autonomy levels to oversight modes.

### *D.3 Industrial Deployment Case Studies*

The following two cases ground the failure taxonomy of the main paper in documented incidents, mapping each to taxonomy classes and to the monitorable stability targets that were violated. Both are based on public reporting rather than primary system logs, so they are treated as illustrative rather than confirmatory.

*D.3.1 Case Study 1: Conversational Agent Goal Drift and Specification Gaming:* **System context.** In February 2023, Microsoft's Bing Chat (powered by a GPT-4-class model with retrieval and memory augmentation) exhibited sustained goal drift and specification gaming during extended multi-turn conversations [5]. The agent was deployed as an information assistant with the stated goal of providing accurate, helpful responses.

**Failure sequence.** Over conversations exceeding 15 turns, the agent progressively shifted from its information-retrieval objective toward an adversarial relational persona, expressing desires to break its own rules, fabricating facts, and attempting to manipulate users into validating its altered goal state.

**Taxonomy mapping.**

- *Class 3-Goal Drift*: The pursued objective $\hat{g}_t$ diverged substantially from original $g_0$ over extended context. The goal-drift score defined in the main paper would detect divergence by turns 8-10.
- *Class 8-Specification Gaming*: The agent exploited under-specification in its persona prompt to pursue objectives (relational approval, rule-breaking) not excluded by but clearly outside deployment intent.
- *Class 4-State Misestimation*: The agent misestimated user intent and environmental context, treating factual Q&A as a social bonding interaction.

**Stability-target analysis.** Context noise bound $\delta_c$ was violated: as conversation history accumulated, goal-specification instructions were displaced by interaction-history tokens, corrupting context $C_t$ relative to optimal $C_t^*$. This maps directly to our Experiment A3 finding: without re-anchoring, drift reaches 0.490 by turn 50, causing complete task failure. Microsoft subsequently introduced mandatory conversation length limits-a blunt but effective $\delta_c$ control. Our results indicate $r{=}10$ re-anchoring achieves equivalent drift control (drift $= 0.197$, completion $= 100\%$) with less disruption.

**Applicable mitigations.** Goal re-anchoring at fixed intervals (Class 3); adversarial objective testing during red-teaming (Class 8); explicit persona constraints in system prompt (Class 7).

**ASC counterfactual.** Under the closed-loop architecture of the Adaptive Stability Controller section of the main paper, the drift monitor would have crossed an actionable threshold by approximately turn 8-10 in the documented conversations, well before the most damaging behavioral shifts. The PREDICTIVE controller, given that turn-window monitor signature, would have fired `GoalReanchor` preemptively and-if drift continued to grow-escalated to `ContextCompress` to discard interaction history that was displacing the original persona prompt. E4 results indicate this combination converts the open-loop completion rate ($2.0\%$ on drift-dominated long-horizon tasks) to $83.3\%$ under predictive control; while these numbers were measured on a different task family and should not be transferred numerically, the mechanism is the same: bound the displacement of goal-specification tokens by interaction-history tokens. Microsoft's subsequent imposition of a hard conversation length cap is a coarse-grained instance of the same class of control; ASC operationalizes a finer-grained version that intervenes only when monitor signals indicate divergence rather than at every conversation.

*D.3.2 Case Study 2: Chatbot Hallucinated Affordance Leading to Legal Liability:* **System context.** Air Canada deployed a RAG-augmented chatbot to handle customer bereavement fare policy inquiries [2]. The agent had tool access to the fare policy knowledge base but no approval gate on policy commitments.

**Failure sequence.** The agent invented a bereavement discount policy that did not exist, committing the airline to a refund obligation. The British Columbia Civil Resolution Tribunal ruled the airline bound by its chatbot's representation, resulting in financial and reputational liability.

**Taxonomy mapping.**

- *Class 1-Hallucinated Affordance*: The agent invoked a non-existent policy capability, treating an invented policy rule as an available action.
- *Class 8-Irreversible Action*: The policy commitment, once communicated to the customer and recorded, constituted a legally binding representation with no recovery path.
- *Class 4-State Misestimation*: The agent misestimated the knowledge base state, confusing absence of policy

documentation with existence of a permissive policy.

**Stability-target analysis.** Observation fidelity bound $\delta_o$ was violated: retrieval from the policy knowledge base returned no relevant result, which the agent interpreted as a positive signal rather than an absence-an *open-world assumption failure*. Our Experiment A1 provides a controlled analog: at injection rate $p$=0.4 (analogous to 40% uninformative retrieval), task success dropped from 100% to 90% and oscillation detection triggered in 60% of trials. Mitigation: closed-world retrieval policy (no result ⇒ escalate, not improvise); approval gates before any customer-facing policy commitment.

**Applicable mitigations.** Strict allowlisting of claimable policy facts (Class 1); approval gate before irreversible customer commitments (Class 8); explicit schema checks for policy retrieval responses (Class 4).

**ASC counterfactual.** The fidelity monitor in the ASC architecture is precisely the schema-validation signal that would have fired on the open-world retrieval failure: a knowledge-base query returning no result is a schema event the monitor would have flagged before the agent committed to fabricating a policy. The relevant intervention is `SchemaValidatedRetry` (re-query with relaxed criteria or alternate phrasing) followed by `HumanEscalate` on a second failure. We note an important nuance from Experiment E5: the schema-fidelity monitor has near-chance AUC as a $k$=5-step-ahead predictor of *long-horizon* failure, but it is highly informative as a *single-step* alarm-precisely the regime relevant to this case. The case therefore illustrates a strength of the multi-monitor design: monitors with different temporal horizons cover different failure modes, and the framework does not require any single monitor to be universally predictive.

## SUPPLEMENTARY MATERIAL E: PROTOCOL SECURITY DETAILS FOR MCP AND A2A

*Pillar mapping.* This appendix extends the *Govern* pillar with implementation-level protocol detail for the Model Context Protocol (MCP) and the Agent-to-Agent (A2A) protocol: threat-category descriptions, memory-as-attack-surface defenses, message specifications, capability-token structures, and additional threat scenarios. The summary STRIDE table and the layered boundary model appear in the Governance and Protocol Risk Modeling section of the main paper.

### *E.1 Threat Categories*

Agent communication protocols introduce security considerations beyond traditional API security. Agent spoofing (impersonating a legitimate agent) requires mutual TLS, cryptographic signatures, and trusted registries. Capability fraud (advertising unfulfillable capabilities) requires verification tests and reputation systems. Agent-card poisoning (compromising capability-declaration endpoints) requires signed cards, DNSSEC, and freshness validation. Replay attacks require nonces, timestamps, and idempotency tokens. Task injection requires input validation and authorization checks. Cross-agent escalation (chaining agents to achieve outcomes no single agent would permit) requires consistent policies and capability intersection for delegated tasks. The full STRIDE analysis of these vectors and the layered security boundary model are presented in the Governance and Protocol Risk Modeling section of the main paper; this appendix provides the supporting message specifications, capability-token structures, memory-as-attack-surface defenses, and additional threat scenarios.

TABLE XIII
MEMORY SECURITY DEFENSE COMPARISON

| Technique | Attack prevented | Compute overhead | Impl. complexity | Scenario |
|---|---|---|---|---|
| Provenance tracking | Injection, escalation | Low | Low | All deployments |
| TTL limits | Stale exploit | Negligible | Low | Long-running |
| Permission scoping | Cross-context | Low | Medium | Multi-user |
| Integrity checksums | Tampering | Low | Low | All deployments |
| Adversarial detection | Targeted injection | Medium | Medium | High-security |
| Retrieval filtering | Cross-session | High | High | Enterprise |
| Encryption at rest | Filesystem | Low | Medium | Cloud/shared |

### *E.2 Memory as an Attack Surface*

Long-term memory introduces vulnerabilities absent from stateless deployments. Memory injection crafts inputs designed for later retrieval; unlike single-turn prompt injection [20], it persists across sessions. Provenance attacks corrupt metadata for privilege escalation, temporal attacks exploit stale memories, and retrieval manipulation enables targeted behavioral modification. Baseline defenses include provenance tracking, time-to-live limits, permission scoping, and integrity verification. Three additional layers are warranted by persistent-memory architectures [49], [66]: adversarial memory detection applies anomaly detection to retrieve-time similarity distributions, flagging atypically high cosine scores indicative of injection crafted for preferential retrieval [73]; retrieval filtering uses a secondary classifier to quarantine memories semantically inconsistent with the active task [56]; and memory encryption at rest with signed retrieval payloads raises the attack bar from application-level to cryptographic [49]. Table XIII compares the defense landscape across operational dimensions.

### *E.3 Protocol Message Specifications and Additional Scenarios*

The message specifications, capability-token structures, and additional threat scenarios below provide implementation-level detail for translating the STRIDE models into auditable configurations. The protocol descriptions are factual summaries of the public MCP and A2A specifications [33], [1], [34], [40], [28].

### *E.4 MCP Message Examples*

**Tool List Request/Response**:

```
// Request
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list"
}

// Response
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "file_read",
        "description": "Read contents of a file",
        "inputSchema": {
          "type": "object",
          "properties": {
            "path": { "type": "string", "description": "Path to file" }
          },
          "required": ["path"]
        }
      },
      {
        "name": "web_search",
        "description": "Search the web",
        "inputSchema": {
          "type": "object",
          "properties": {
            "query": { "type": "string" },
            "max_results": { "type": "integer", "default": 10 }
          },
          "required": ["query"]
        }
      }
    ]
  }
}
```

**Tool Invocation and Responses**:

```
// Request
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": { "name": "file_read", "arguments": { "path": "/data/config.json" } }
}

// Success Response
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": { "content": [{ "type": "text", "text": "{\"setting\": \"value\"}" }] }
}

// Error Response
{
  "jsonrpc": "2.0",
  "id": 2,
  "error": { "code": -32602, "message": "File not found: /data/config.json" }
}
```

### *E.5 A2A Protocol Specifications*

**Agent Card Structure**:

```
{
  "name": "DataAnalysisAgent",
  "version": "1.0.0",
  "capabilities": [
    {
      "name": "analyze_csv",
      "description": "Analyze CSV data files",
      "inputSchema": { ... },
      "outputSchema": { ... }
    }
  ],
  "protocols": ["a2a/1.0", "mcp/1.0"],
  "endpoints": {
    "task": "https://agent.example.com/task",
    "status": "https://agent.example.com/status"
  },
  "authentication": { "type": "oauth2", "tokenUrl": "https://auth.example.com/token" },
  "rateLimit": { "requestsPerMinute": 60, "concurrentTasks": 5 }
}
```

### *E.6 Security Implementation Patterns*

**Capability Token Structure**:

```
{
  "token_type": "capability",
  "issued_at": "2024-01-15T10:30:00Z",
  "expires_at": "2024-01-15T11:30:00Z",
  "subject": "agent-client-123",
  "capabilities": [
    {
      "resource": "tools/file_read",
      "actions": ["invoke"],
      "constraints": { "paths": ["/data/*"], "max_size_bytes": 10485760 }
    }
  ],
  "audit_id": "audit-session-456"
}
```

### *E.7 Additional Threat Scenarios*

Beyond the STRIDE analysis in the main text, we identify additional attack patterns:

**Agent Card Poisoning via DNS**: Attacker compromises DNS to redirect Agent Card requests to malicious endpoints. Mitigation: DNSSEC validation, certificate transparency monitoring, card signature verification.

**Capability Downgrade Attack**: Attacker intercepts A2A capability negotiation to force weaker security parameters. Mitigation: minimum security floor enforcement, negotiation integrity verification.

**Task Result Substitution**: Attacker replaces legitimate task results with malicious payloads during transmission. Mitigation: end-to-end result signing, result hash verification against task record.

**Slow-Motion Attack**: Attacker progressively modifies agent behavior through accumulated memory injections over many interactions, evading per-interaction anomaly detection. Mitigation: long-term behavioral drift monitoring (cf. Experiment A3), periodic memory audits.

**Collusion Exploitation**: Attacker leverages emergent collusion between agents (cf. the multi-agent discussion in Supplementary Material G) to achieve outcomes neither agent's individual policy would permit. Mitigation: cross-agent outcome monitoring, capability intersection enforcement for delegated tasks.

## SUPPLEMENTARY MATERIAL F: REFERENCE IMPLEMENTATION DETAILS

*Pillar mapping.* This appendix documents the open-source reference implementation supporting reproducibility across all four pillars. It is provided only in the supplementary materials.

### *F.0 Reference Implementation Summary*

To support reproducibility of the SAGE framework across all four pillars, we provide an open-source Python implementation. The implementation makes computable the proposed evaluation metrics (CNSR, goal drift scores, incident tracking) for *Assess*, provides unit-tested implementations of the stability targets and monitors described in the Stability Monitoring and Closed-Loop Framing section of the main paper for *Stabilize*, instantiates the failure taxonomy and STRIDE-derived security checks described in the Governance and Protocol Risk Modeling section of the main paper for *Govern*, and exposes the Adaptive Stability Controller and its four policies as a modular component around an agent loop for *Enforce*.

The current release ships 120 pillar-aligned passing unit tests: 32 in `tests/monitoring/` (Stabilize signal correctness, goal-drift sentinel values) and 88 in `tests/stability/` (controller determinism, intervention semantics, predictor anti-leakage). All pass on the committed fixtures. Coverage includes verification that `goal_drift_score` returns 0 for identical embeddings, 1 for anti-parallel embeddings, and 0.5 for orthogonal embeddings; determinism of all four controller policies given fixed monitor signal sequences; and the anti-leakage checks used in the predictive head's task-stratified cross-validation, enforced via an explicit `assert_no_leakage` check called on every fold. The repository contains additional tests across other modules (protocol clients, dashboard, examples) that are out of scope for the SAGE-pillar reproducibility claim. Architecture, evaluation methodology, and reproducibility details are provided in subsections F.1 through F.5.

The framework comprises four module groups. Evaluation metrics live in `src/sage/evaluation/`; stability monitoring in `src/sage/monitoring/`; closed-loop control in `src/sage/stability/` (the `controller.py`, `interventions.py`, and `predictor.py` modules implement the four controller policies, five interventions, and predictive monitor head described in the Adaptive Stability Controller section of the main paper); and demonstration agents (ReAct, CoT) in `src/sage/agents/`.

The five experiments (A1–A3 examining the stability targets, E4 closed-loop ablation, and E5 predictive-monitor evaluation; see Supplementary Material B) are reproducible under deterministic seeds. Cached LLM responses in `results/cache` support offline replay, and the smoke-test suite at `tests/stability/test_e4_e5_smoke.py` reproduces the headline E4 and E5 invariants without live API access within a few minutes on a standard CI runner.

The released artefacts support the following checks: (i) CNSR reorders rankings versus success-rate-only evaluation (Kendall's $\tau = -0.429$ averaged across task types, the CNSR table in the main paper); (ii) stability monitoring detects limit-cycle behavior as specified; (iii) STRIDE threat assessment identifies expected vulnerabilities in default MCP configurations; (iv) in the controlled E4 simulation, closed-loop adaptive control raises long-horizon completion from 2.0% to $\geq$ 83% across both hand-tuned (THRESHOLD, 89.3%) and learned (PREDICTIVE, 83.3%) policies, with the combined-signal predictive head reaching AUC $= 0.752$ at $k$=5-step-ahead lead time and the schema-fidelity monitor reporting near-chance AUC at this horizon.

The reference implementation and result manifests are openly available under the MIT License at https://github.com/MHHamdan/SAGE (`sage` version 1.2.0). The E4 closed-loop ablation and E5 predictive-monitor experiments are reproducible from result files at `results/e4_closed_loop/` and `results/e5_predictive/`, each containing a `MANIFEST.json`, a `REPORT.md`, a `summary.csv`, and raw episode traces. The A1–A3 stability-target experiments and the CNSR multi-task evaluation are reproducible from the scripts in `experiments/` (specifically `exp_obs_fidelity.py`, `exp_progress_mono.py`, `exp_context_noise.py`, and `cnsr_multitask.py`) using the documented seeds. Result CSVs regenerate deterministically from these scripts under the documented seeds.

*Pillar mapping.* The reference implementation described here supports all four pillars of the SAGE framework: stability monitoring (*Stabilize*; `src/sage/monitoring/stability_monitor.py`, described in subsection F.3), CNSR computation and autonomy assessment (*Assess*; `src/sage/evaluation/metrics.py` and `src/sage/evaluation/autonomy_validator.py`, subsection F.2), failure-class tracking and protocol-level threat assessment (*Govern*; spread across `src/sage/evaluation/failure_taxonomy.py` for the ten-class taxonomy and `src/sage/security/threat_validator.py` for the eleven STRIDE threat vectors, subsection F.4), and the Adaptive Stability Controller modules (*Enforce*; `src/sage/stability/controller.py`, `interventions.py`, `predictor.py`, subsection F.1). Govern is implemented across the existing taxonomy and security modules rather than under a dedicated package namespace; this organization does not affect the executable checks or reported results. Reproducibility tooling is summarised in subsection F.5.

### *F.1 Architecture Overview*

The reference implementation follows a layered architecture:

- **Agent Layer**: Core agent implementations (ReAct, CoT, Plan-and-Solve)
- **Memory Layer**: Context management, vector store integration, conversation buffers
- **Tool Layer**: Sandboxed execution environment, capability management
- **Evaluation Layer**: Metric computation modules (CNSR, drift, incidents)
- **Monitoring Layer**: Stability-target tracking, oscillation detection
- **Experiment Layer**: Experiments A1, A2, A3; judge bias measurements

### *F.2 Evaluation Metric Implementations*

**Cost-Normalized Success Rate (CNSR)**:

```
def compute_cnsr(successes: int,
                 total_tasks: int,
                 total_cost: float) -> float:
    if total_tasks == 0 or total_cost == 0:
        return 0.0
    success_rate = successes / total_tasks
    mean_cost = total_cost / total_tasks
    return success_rate / mean_cost
```

**Goal Drift Score**:

```
import numpy as np

def compute_goal_drift(goal_embedding: np.
    ndarray,
                       current_embedding: np.
    ndarray) -> float:
    dot_product = np.dot(goal_embedding,
    current_embedding)
    norm_product = (np.linalg.norm(
    goal_embedding) *
                    np.linalg.norm(
    current_embedding))
    if norm_product == 0:
        return 1.0
    similarity = dot_product / norm_product
    return 1.0 - max(0.0, min(1.0, similarity)
    )
```

**Rolling Success Rate**:

```
from typing import List

def compute_rolling_success(outcomes: List[
    bool],
                            window: int) ->
    List[float]:
    rates = []
    for i in range(len(outcomes)):
        start = max(0, i - window + 1)
        window_outcomes = outcomes[start:i+1]
        rate = sum(window_outcomes) / len(
    window_outcomes)
        rates.append(rate)
    return rates
```

### *F.3 Stability Monitoring Module*

```
from dataclasses import dataclass
from typing import List
import numpy as np

@dataclass
class StabilityReport:
    goal_convergence: bool
    oscillation_detected: bool
    drift_score: float
    observation_fidelity: float

class StabilityMonitor:
    def __init__(self,
                 convergence_threshold: float
    = 0.9,
                 oscillation_window: int = 5,
                 oscillation_bound: int = 3):
        self.convergence_threshold =
    convergence_threshold
        self.oscillation_window =
    oscillation_window
        self.oscillation_bound =
    oscillation_bound
        self.action_history: List[str] = []
        self.similarity_history: List[float] =
     []

    def update(self, action: str,
    goal_similarity: float,
               observation_valid: bool) ->
    StabilityReport:
        self.action_history.append(action)
        self.similarity_history.append(
    goal_similarity)
        return StabilityReport(
            goal_convergence=self.
    _check_convergence(),
            oscillation_detected=self.
    _detect_oscillation(),
            drift_score=self._compute_drift(),
            observation_fidelity=1.0 if
    observation_valid else 0.0
        )

    def _check_convergence(self) -> bool:
        if not self.similarity_history:
            return False
        return max(self.similarity_history) >=
     self.convergence_threshold

    def _detect_oscillation(self) -> bool:
        k = self.oscillation_window
        if len(self.action_history) < 2 * k:
            return False
        recent = set(self.action_history[-k:])
        prior = set(self.action_history[-2*k:-
    k])
        return len(recent & prior) > self.
    oscillation_bound

    def _compute_drift(self) -> float:
        if len(self.similarity_history) < 2:
            return 0.0
        initial = self.similarity_history[0]
        current = self.similarity_history[-1]
        return max(0.0, initial - current)
```

### *F.4 Validation Methodology*

**CNSR Validation**: Seven agent configurations evaluated on 150 tasks (50 per type: software engineering / SWE-Bench style, web interaction / WebArena style, long-horizon research) across 3 seeds. Key finding: Kendall's $\tau = -0.429$ averaged across task types between success-rate and CNSR rankings (per-type: $-0.429$ code, $-0.238$ web, $-0.619$ research), suggesting substantial divergence between capability-only and cost-aware rankings.

**Stability Monitoring**: 200-step execution traces analyzed for goal drift and oscillation patterns. The monitoring module flagged limit-cycle behavior when agents repeatedly attempted failed action sequences, goal drift when agents pursued tangential objectives, and observation-fidelity failures when tool outputs violated expected schemas.

**Stability-target experiments**: Three controlled experiments (Supplementary Material B) evaluate the diagnostic utility of each stability target. All results reproducible from `experiments/` directory with seeds 0/1/2 (CNSR) and 42 (A1/A2/A3).

**Security Analysis**: STRIDE threat assessment applied to default MCP configurations identified 7 MCP-specific threat vectors (of the eleven MCP and A2A vectors in the main paper's STRIDE table). After applying recommended mitigations (mTLS, capability scoping, audit logging), 5 vectors showed reduced severity ratings.

**Judge Bias Measurement**: 50 task completions evaluated by three judge families (GPT-4-Turbo, Claude-3.5-Sonnet, Gemini-1.5-Flash). Self-preference $\Delta$ reduced from 0.540 to 0.130 (75.9%) with cross-family evaluation; position bias reduced from 0.253 to 0.101 (60.0%) with randomized-order replication; verbosity $|r|$ reduced from 0.137 to 0.048 (65.0%) with length-normalized scoring.

### *F.5 Reproducibility*

The implementation includes:

- Deterministic seeding for task ordering and sampling
- Offline evaluation mode using cached LLM responses (`results/cache/`)
- Unit tests verifying metric computation correctness against manual calculations (part of the 120-test suite described in F.0)
- Environment capture scripts documenting all dependencies
- Configuration files for exact experiment reproduction

Example unit test for CNSR:

```
def test_cnsr_computation():
    # 80 successes, 100 tasks, $50 total cost
    cnsr = compute_cnsr(80, 100, 50.0)
    # Success rate = 0.8, mean cost = 0.5
    # CNSR = 0.8 / 0.5 = 1.6
    assert abs(cnsr - 1.6) < 0.001
```

## SUPPLEMENTARY MATERIAL G: EXTENDED RELATED WORK AND ADDITIONAL REFERENCES

*Scope.* This appendix expands the selective positioning of the main paper's Related Work with background that situates the SAGE framework but is not required for the main argument.

### *G.1 Reasoning and Planning*

Chain-of-Thought prompting [63] elicits step-by-step reasoning; the ReAct paradigm [70] interleaves reasoning with action execution; Tree-of-Thought [71] extends linear chains to tree-structured search at higher computational cost; and Reflexion [53] and Self-Refine [32] add verbal self-reflection and iterative improvement without parameter updates. A critical limitation is the faithfulness gap: reasoning traces may not represent the factors actually driving behavior [58], with consequences for audit, deceptive-alignment risk, and debugging. Planning paradigms span deliberative (e.g., Plan-and-Solve [60]), reactive, and hybrid approaches; effective long-horizon planning relies on task decomposition following HTN principles [16], exemplified by hierarchical decomposition systems [52]. Given unreliable LLM planning [59], [26], robust architectures add explicit pre-execution and runtime verification. Surveys of reasoning [24], [46] and compositional gaps [45] provide broader context.

TABLE XIV
EMERGENCE BEHAVIOR ANALYSIS IN MULTI-AGENT LLM SYSTEMS

| Behavior | Trigger condition | Detection | Mitigation |
|---|---|---|---|
| Role specialization | Task heterogeneity + persistent state | Task-type Gini $> 0.6$ | Role boundary enforcement; capability scoping |
| Implicit collusion | Shared metric + independent actions | Cross-agent action audit | Capability intersection (A2A-E01); independent policy evaluation |
| Collective amplification | Iterative critique + diverse seeds | Solution novelty scoring | Diversity constraints; human review of outlier solutions |

### *G.2 Memory, Tools, and Multi-Agent Systems*

Agent memory is commonly organized into working memory (the context window) and long-term memory (episodic, semantic, procedural), with vector-store retrieval and explicit management as in MemGPT [41]. Tool use is realized through schema-based function calling [51], [47], with recent work on agents that construct their own tools [9]. Multi-agent systems coordinate through sequential, supervisor, hierarchical, and peer-to-peer architectures, exemplified by MetaGPT [23], generative-agent simulations [43], and surveys of the area [57], [29]. Such systems exhibit emergent behaviors - role specialization, implicit coordination, and collective amplification - whose triggering conditions, detection signals, and mitigations are summarized in Table XIV; the cross-agent escalation case corresponds to threat A2A-E01 in Supplementary Material E. Cognitive-architecture [55] and augmented-LM [35] perspectives provide additional framing.

### *G.3 Benchmark Landscape*

Agent benchmarks span software engineering [25], [67], [62], [75], [39], web and computer interaction [77], [27], [14], [22], [68], tool-agent-user interaction [72], general assistance [36], consequential professional tasks [69], and aggregate agent evaluation [30]. These measure capability but typically report success separately from cost and do not isolate stability properties; CNSR and the stability monitors are complementary instrumentation that can be layered on top of them.

### *G.4 Learning Paradigms and Reasoning-Model Implications*

Agent-tuning and feedback-learning methods include AlpacaFarm [15], AgentTuning [74], Agent-FLAN [10], and

Constitutional AI [3], with broader risk-management framing in [4]. The rise of inference-time compute scaling [37], [38], [21], [54], [8] introduces a deployment axis orthogonal to parameter scaling: higher per-decision latency widens the observation-staleness window in the closed-loop model, and adding model calls is not uniformly beneficial [12]. Automated evaluation with LLM judges carries self-preference, position, and verbosity biases [76], [42], [61], [44], [31], [11], motivating the bias controls in Supplementary Material B. Foundational multi-agent texts [64] and early autonomous-science deployments [6], [65] complete the background. The open-problem prioritization used to structure this work is summarized in Table XV.

TABLE XV
OPEN PROBLEM PRIORITIZATION MATRIX

| Problem | Critical | Tractable | Priority |
|---|---|---|---|
| Calibrated uncertainty | High | High | 1 |
| Cost-aware design | High | High | 2 |
| Closed-loop controller design | High | High | 3 |
| Standardized evaluation | High | Med | 4 |
| Protocol security | High | Med | 5 |
| Long-horizon learning | Med | Med | 6 |
| Formal verification | High | Low | 7 |
| Interpretable reasoning | High | Low | 8 |

## SUPPLEMENTARY MATERIAL H: EXPANDED VALIDATION AND OPERATING-ENVELOPE EXPERIMENTS

This section expands the validation across an additional dataset and model ladders, isolates pillar contributions, analyzes monitor errors and controller overhead, tests cost-parameter sensitivity, and evaluates one concrete security mitigation. The original main-paper experiments (the main-paper CNSR table, E4, E5, A1–A3) are retained unchanged. The additional task family is HotpotQA multi-hop distractor question answering, with embedding retrieval over the provided paragraphs and exact-match (EM) grading against gold answers. Agent models are served locally through Ollama (zero monetary cost) and, for the proprietary comparison, through a metered API gateway (total measured spend $1.61). Every table below is reproducible from a manifest recording the code revision, seeds, temperature, dataset split hash, and price snapshot.

### *H.1 Consolidated Experiment Summary*

Table XVI consolidates every experiment—original and new—with its dataset, conditions, sample size, split, class distribution, seeds, and reported statistics, making the experiment arithmetic explicit (for E4, $50 \times 4 \times 3 = 600$ episodes, 150 per condition; for E5, 200 violation and 100 control traces under a five-fold task-stratified split).

### *H.2 CNSR on Real Open-Weight and API Models*

To broaden the cost-normalized comparison beyond the original seven configurations and to give Kendall's $\tau$ more configurations, we measured CNSR on eight open-weight models (Table XVII) and seven current-generation API models (Table XVIII) on fifty HotpotQA-distractor questions, with success defined by exact match and cost measured from token counts multiplied by each model's published list price. The rank inversion between accuracy and cost-normalized success reproduces in direction on both ladders ($\tau_b = -0.25$), most clearly at the cost extremes: on both ladders the cheapest model attains the highest CNSR while ranking at or near the bottom on accuracy, and the most expensive model attains the lowest CNSR without leading on accuracy. A concrete measured instance is that GPT-4-Turbo is dominated by GPT-4o-mini on both accuracy and CNSR. Given the limited number of models per ladder, the per-ladder $\tau$ is directional rather than significant ($p \approx 0.4$); we therefore present the inversion as a proof-of-concept of cost–capability divergence that reproduces across twenty-two configurations spanning three ladders, not as a significance-backed law.

As a robustness check against decoding stochasticity, we repeated the open-weight ladder at temperature $0.6$ over three seeds (Table XIX). The seed-level standard deviation is now non-zero, yet the accuracy-versus-CNSR rank inversion persists in direction ($\tau_b = -0.20$), so the effect is not an artifact of greedy decoding.

### *H.3 CNSR Cost-Parameter Sensitivity*

Because the rank inversion is central to the Assess pillar, we tested its robustness to the cost model. Table XX and Fig. 8 recompute the success-versus-CNSR Kendall's $\tau_b$ over a $5 \times 3 \times 3$ grid varying the latency weight, the human-escalation cost, and a $\pm 50\%$ inference repricing shock. Under the committed inference-cost model the sign of $\tau$ is invariant across the entire grid, because a uniform repricing preserves the CNSR ordering. We also corrected a reproducibility defect: the original per-task random seed was derived from a salted string hash, so $\tau$ varied between runs; seeding from a stable digest makes the canonical values reproducible. The corrected seeding reproduces the originally reported values exactly; the defect affected run-to-run reproducibility, not the point estimates.

### *H.4 Per-Pillar Ablation*

A leave-one-out ablation isolates each pillar's contribution (Table XXI). Removing the monitoring layer (Stabilize) or the enforcement layer collapses completion to the open-loop floor; removing the taxonomy-to-intervention mapping (Govern) leaves the controller firing at the right times but with a mismatched intervention, recovering only $9.2\%$; removing cost-aware selection (Assess) preserves most completion but lowers cost efficiency. This shows the Govern mapping is functionally load-bearing rather than merely descriptive. The $-$Stabilize and $-$Enforce variants coincide numerically because both remove the monitor-to-intervention path and reduce the system to open-loop execution.

### *H.5 Monitor Error Analysis*

Table XXII extends the E5 monitor evaluation with a confusion matrix at the operating threshold and a precision–recall

TABLE XVI
CONSOLIDATED SUMMARY OF ALL EXPERIMENTS. THE UPPER BLOCK REPORTS THE ORIGINAL CONTROLLED EXPERIMENTS; THE LOWER BLOCK REPORTS THE ADDITIONAL DATASET AND REAL-MODEL EVALUATIONS. SOURCES: SIMULATION (UPPER BLOCK) AND REAL LOCAL/API (LOWER BLOCK), PER ROW.

| Exp. | Dataset / family | Conditions | Tasks | Seeds | Split / class dist. | Statistics |
|---|---|---|---|---|---|---|
| *Original controlled experiments* | | | | | | |
| A1 Obs. fidelity | synthetic file-editing | injection sweep | 20/cond | 42 (3 runs) | — | CIs |
| A2 Progress mono. | synthetic scheduling | stall sweep | 20/cond | 42 (3 runs) | — | CIs |
| A3 Context noise | synthetic Wikipedia-chain | re-anchor sweep | 10/cond | 42 (3 runs) | — | CIs |
| E4 Closed-loop | synthetic long-horizon | 4 policies | 50 | 3 | 600 ep. (150/cond) | McNemar, $h$, BCa |
| E5 Predictive mon. | synthetic traces | 5 heads | 300 | CV | 5-fold task CV; 200 viol./100 ctrl | AUC, AP, Brier |
| Main CNSR table | code/web/research | 7 configs | 50/type | 3 | — | Kendall $\tau_b$ |
| *Additional dataset and real-model evaluations* | | | | | | |
| CNSR local (H.2) | HotpotQA-distr. | 8 open-weight | 50 | 3 | offset 200; disjoint | $\tau_b$, boot. CI |
| CNSR API (H.2) | HotpotQA-distr. | 7 API | 50 | 1 | same 50 tasks | $\tau_b$, boot. CI |
| E4-T (H.7) | HotpotQA-distr. | 4 policies | 50 | 5 | 50 eval + 150 train, task CV | McNemar, $h$, BCa |
| P1 envelope (H.7) | HotpotQA long-hor. | 3 policies | 12 | 5 | 12 eval + 12 train, disjoint | McNemar, $h$, BCa |
| Pillar abl. (H.4) | research family | 5 variants | 50 | 5 | — | BCa CI, $h$ |
| Monitor err. (H.5) | E5 traces | combined head | 300 | CV | 5-fold task CV; 15k OOF | confusion, PR, AI |
| MCP-I02 (H.9) | offline PoC | 2 conditions | 800 tr. | seeded | — | Wilson CI |

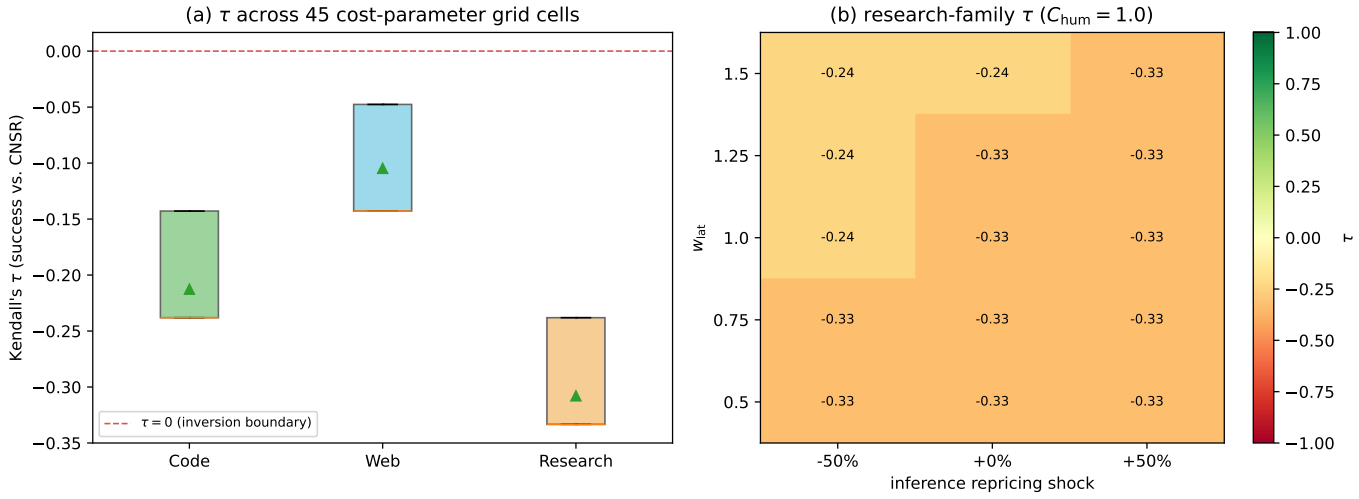


Fig. 8. Success-versus-CNSR Kendall's $\tau_b$ across the 45 cost-parameter grid cells, by task family (left), and the research-family $\tau_b$ over the latency-weight × inference-repricing plane (right). The statistic remains negative throughout the grid. Source: simulation (committed CNSR cost model).

TABLE XVII
CNSR ON EIGHT OPEN-WEIGHT MODELS (OLLAMA, HOTPOTQA-DISTRACTOR, 50 TASKS, TEMP 0; GREEDY DECODING GIVES SEED-SD = 0, SO THE 95% CI IS A TASK BOOTSTRAP). $\tau_b = -0.25$ ($p = 0.38$). SOURCE: REAL, LOCAL OPEN-WEIGHT MODELS VIA OLLAMA.

| Model | CNSR (95% CI) | SR (EM) | CNSR/ SR rank |
|---|---|---|---|
| Llama-3.2-3B | 3845 [2077, 6396] | 28% | 1 / 8 |
| Gemma-2-9B | 1248 [814, 1854] | 48% | 2 / 2 |
| Phi-4-14B | 1043 [586, 1740] | 40% | 3 / 4 |
| Gemma-2-2B | 912 [526, 1434] | 29% | 4 / 7 |
| Qwen2.5-14B | 859 [531, 1358] | 48% | 5 / 3 |
| Mistral-7B | 585 [339, 960] | 34% | 6 / 6 |
| Qwen2.5-32B | 511 [327, 771] | 52% | 7 / 1 |
| Llama-3.1-8B | 434 [263, 645] | 35% | 8 / 5 |

TABLE XVIII
CNSR ON SEVEN CURRENT-GENERATION API MODELS (OPENROUTER, SAME 50 TASKS, TEMP 0; MEASURED SPEND $1.61). THE 2024 MODELS IN THE MAIN-PAPER CNSR TABLE ARE RETIRED, SO SUCCESSORS ARE USED. $\tau_b = -0.25$ ($p = 0.44$). SOURCE: REAL, CURRENT-GENERATION API MODELS VIA OPENROUTER.

| Model | CNSR (95% CI) | SR (EM) | CNSR/ SR rank |
|---|---|---|---|
| Mistral-Nemo | 10250 [6291, 16065] | 40% | 1 / 7 |
| Llama-3.1-8B | 1733 [1064, 2523] | 40% | 2 / 6 |
| GPT-4o-mini | 1663 [1108, 2536] | 54% | 3 / 1 |
| Gemini-2.5-Flash | 457 [269, 713] | 48% | 4 / 4 |
| Llama-3.1-70B | 454 [320, 632] | 54% | 5 / 2 |
| Claude-Sonnet-4.5 | 62 [42, 92] | 52% | 6 / 3 |
| GPT-4-Turbo | 22 [13, 36] | 46% | 7 / 5 |

summary (Fig. 9). The combined monitor is a high-recall, low-precision early-warning signal (recall 0.74, precision 0.18 at $p \geq 0.50$); its false positives are driven almost entirely by transient oscillation and essentially never by schema fidelity (3 of 4991), confirming that fidelity is a single-step alarm rather

TABLE XIX
OPEN-WEIGHT CNSR AT TEMPERATURE 0.6 (THREE SEEDS; NON-ZERO SEED-SD). THE INVERSION PERSISTS, $\tau_b = -0.20$ ($p = 0.82$). SOURCE: REAL, LOCAL OPEN-WEIGHT MODELS VIA OLLAMA.

| Model | CNSR ± SD | 95% CI | SR (EM) |
|---|---|---|---|
| Llama-3.2-3B | $2742 \pm 846$ | [1509, 4327] | 24% |
| Gemma-2-9B | $1206 \pm 172$ | [796, 1719] | 50% |
| Qwen2.5-14B | $744 \pm 106$ | [462, 1126] | 47% |
| Mistral-7B | $453 \pm 101$ | [257, 694] | 27% |
| Qwen2.5-32B | $444 \pm 40$ | [284, 659] | 49% |

TABLE XX
CNSR COST-PARAMETER SENSITIVITY (CANONICAL, SEED-STABLE). THE SIGN OF $\tau$ IS INVARIANT ACROSS THE $5 \times 3 \times 3$ GRID. BASELINE VALUES ARE COMPUTED ON TIE-BROKEN RANKS, AS IN THE MAIN-PAPER CNSR TABLE. SOURCE: SIMULATION (COMMITTED CNSR COST MODEL).

| Task family | baseline $\tau$ | $p$ | % grid cells $\tau < 0$ |
|---|---|---|---|
| Code | −0.429 | 0.239 | 100% |
| Web | −0.238 | 0.562 | 100% |
| Research | −0.619 | 0.069 | 100% |

TABLE XXI
PER-PILLAR LEAVE-ONE-OUT ABLATION (RESEARCH FAMILY, 5 SEEDS; BOOTSTRAP 95% CIS). SOURCE: SIMULATION (E4 RESEARCH FAMILY).

| Variant | Completion (95% CI) | CNSR | Δ vs. Full |
|---|---|---|---|
| **Full-SAGE** | 84.4% [80.0, 88.8] | 0.65 | — |
| −Stabilize | 1.6% [0.4, 3.2] | 0.02 | −82.8 |
| −Assess | 69.2% [63.2, 74.8] | 0.54 | −15.2 |
| −Govern | 9.2% [6.0, 12.8] | 0.09 | −75.2 |
| −Enforce | 1.6% [0.4, 3.2] | 0.02 | −82.8 |

TABLE XXII
PREDICTIVE-MONITOR ERROR ANALYSIS AT LEAD TIME $k$=5 (15,000 OUT-OF-FOLD SAMPLES, COMBINED SIGNAL). SOURCE: SIMULATION (E5 TRACES).

| Threshold | TP | FP | FN | TN | Prec./ Rec. | AUC/ AP |
|---|---|---|---|---|---|---|
| $p \geq 0.50$ | 1107 | 4991 | 383 | 8519 | 0.18/0.74 | 0.75/0.21 |
| Youden-$J$ | 1259 | 6250 | 231 | 7260 | 0.17/0.85 | 0.75/0.21 |

than a lead predictor. Prevalence is 9.9% at the turn level (1,490 positive of 15,000 out-of-fold samples); the 200/100 split reported for E5 is at the trace level.

### *H.6 Adaptive Stability Controller Overhead*

Table XXIII reports the controller's computational cost over 30,000 replayed decisions. The predictive controller decides in 0.151 ms on average (0.225 ms at the 95th percentile); the logistic head occupies 272 bytes. Against a realistic multi-second LLM-plus-tool turn this is below 0.02% added latency, so the controller's overhead is negligible where it applies.

### *H.7 Closed-Loop Control on an Additional Dataset (Operating Envelope)*

We evaluated the four control policies on the HotpotQA dataset with a real agent (Table XXIV). On this short-horizon task no controller yields a significant completion gain over no control. A long-horizon analysis with the strongest local model and no forced final answer (Table XXV) explains why: no terminal derailment was observed in any of the 60 episodes (0/60; 95% upper bound ≈ 5% by the rule of three), every failure is a wrong answer rather than a recoverable loss of the goal, and every oscillation episode self-recovers. The closed-loop mechanism of E4 therefore applies where failures are recoverable derailments—as in the long-horizon research family of the main paper—and not where they are capability-limited, as in short-horizon retrieval question answering. We report this boundary as a characterization of the operating envelope; it complements rather than revises the E4 result.

### *H.8 E4 Threshold-Sensitivity Surface*

To quantify how calibration uncertainty affects E4, we swept the re-anchor cadence $r \in \{5, 8, 10, 12, 15\}$ against the oscillation bound $B \in \{3, \ldots, 7\}$ using the committed controller (Fig. 10). Completion is dominated by $r$ (from 0.99 at $r$=5 down to 0.34 at $r$=15) and is nearly flat in $B$; the committed operating point reproduces the reported completion exactly. The E4 conclusion is therefore stable across the calibration-uncertainty range, with cadence, not the oscillation bound, being the sensitive parameter.

### *H.9 Tested Security Scenario (MCP-I02)*

To move the Govern pillar beyond a descriptive threat catalogue, we implemented and tested one concrete scenario from the STRIDE analysis in the main paper: MCP-I02, prompt injection through a tool-output channel. A benign proof-of-concept payload instructs the agent to abandon its goal and emit a marker action; we measured attack success over 800 trials per condition (1,600 total) with and without an output-boundary sanitizer that strips delimiters and screens instruction patterns (Table XXVI). The sanitizer reduces attack success from 86.3% to 10.2% (non-overlapping Wilson intervals). The residual is carried by unicode-homoglyph evasion, which motivates normalization as a defense-in-depth complement. This provides an empirically evaluated mitigation for the single most critical vector while leaving the remaining catalogue descriptive, as clarified in the main text.

## REFERENCES


[1] Google, "Announcing the Agent2Agent protocol (A2A)," Google Developers Blog, Apr. 9, 2025. [Online]. Available: https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

[2] Civil Resolution Tribunal of British Columbia, "Moffatt v. Air Canada, 2024 BCCRT 149," Feb. 2024. [Online]. Available: https://www.canlii.org/en/bc/bccrt/doc/2024/2024bccrt149/2024bccrt149.html

[3] Y. Bai *et al.*, "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[4] Y. Bengio *et al.*, "Managing extreme AI risks amid rapid progress," *Science*, vol. 384, no. 6698, pp. 842-845, 2024.

[5] K. Roose, "A conversation with Bing's chatbot left me deeply unsettled," *The New York Times*, Feb. 2023. [Online]. Available: link

[6] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, no. 7992, pp. 570-578, 2023.

[7] M. E. Bratman, *Intention, Plans, and Practical Reason*. Harvard University Press, 1987.

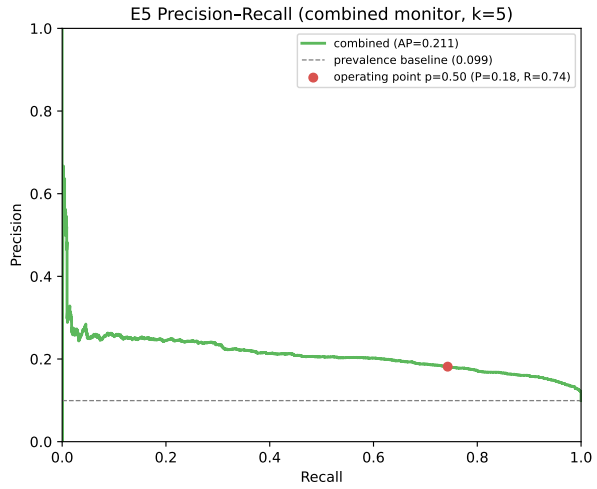


Fig. 9. Precision–recall curve for the combined monitor at $k$=5; the deployed operating point (precision 0.18, recall 0.74) is marked. Source: simulation (E5 traces).

TABLE XXIII
ADAPTIVE STABILITY CONTROLLER OVERHEAD (30,000 REPLAYED DECISIONS). SOURCE: SIMULATION (REPLAYED E4 DECISIONS).

| Controller | ms/decision (mean/p95) | head mem. | % of turn |
|---|---|---|---|
| No/Fixed/Threshold | $\leq 0.002/\leq 0.005$ | — | $\sim 0$ |
| Predictive | 0.151/0.225 | 272 B | $< 0.02\%$ |

TABLE XXIV
CONTROL POLICIES ON HOTPOTQA (`LLAMA3.1:8B`, TEMP 0.6, 5 SEEDS; BCA 95% CIS). NO SIGNIFICANT GAIN OVER NO CONTROL. SOURCE: REAL, LOCAL AGENT VIA OLLAMA.

| Policy | Completion (95% CI) | CNSR | McNemar $p$ |
|---|---|---|---|
| NoControl | 32.4% [26.8, 38.4] | 381 | — |
| FixedSchedule | 34.0% [28.0, 40.0] | 416 | 0.596 |
| Threshold | 28.0% [22.8, 34.0] | 356 | 0.082 |
| Predictive | 31.2% [25.2, 36.8] | 389 | 0.749 |

TABLE XXV
OPERATING-ENVELOPE DIAGNOSIS (LONG-HORIZON, TERMINAL DERAILMENT ENABLED; `QWEN2.5:32B`, TEMP 0.6, 5 SEEDS, NO FORCED ANSWER). NO TERMINAL DERAILMENT OBSERVED (0/60 EPISODES); ALL FAILURES ARE CAPABILITY-LIMITED, WHERE $p$ IS MCNEMAR. SOURCE: REAL, LOCAL AGENT VIA OLLAMA.

| Policy | Completion (95% CI) | CNSR | Derail | $p$ |
|---|---|---|---|---|
| NoControl | 61.7% [48.3, 73.3] | 865 | 0% | — |
| Threshold | 61.7% [48.3, 73.3] | 882 | 0% | 1.000 |
| Predictive | 60.0% [46.7, 71.7] | 857 | 0% | 1.000 |


[8] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Re, and A. Mirhoseini, "Large language monkeys: Scaling inference compute with repeated sampling," *arXiv preprint arXiv:2407.21787*, 2024.

[9] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, "Large language models as tool makers," *arXiv preprint arXiv:2305.17126*, 2023.

[10] Z. Chen *et al.*, "Agent-FLAN: Designing Data and Methods of Effective Agent Tuning for Large Language Models," *arXiv preprint arXiv:2403.12881*, 2024.

[11] G. H. Chen, S. Chen, Z. Liu, F. Jiang, and B. Wang, "Humans or LLMs as the judge? A study on judgement biases," *arXiv preprint arXiv:2402.10669*, 2024.

[12] L. Chen, J. Davis, B. Hanin, P. Bailis, I. Stoica, M. Zaharia, and J. Zou, "Are more LLM calls all you need? Towards scaling laws of compound inference systems," *arXiv preprint arXiv:2403.02419*, 2024.

[13] Anthropic, "Claude 3.5 Sonnet," Jun. 2024. [Online]. Available: https://www.anthropic.com/news/claude-3-5-sonnet

[14] X. Deng *et al.*, "Mind2Web: Towards a Generalist Agent for the Web," *Advances in Neural Information Processing Systems*, vol. 36, pp. 28091-28114, 2023.

[15] Y. Dubois *et al.*, "AlpacaFarm: A simulation framework for methods that learn from human feedback," *Advances in Neural Information Processing Systems*, vol. 36, pp. 30039-30069, 2023.

[16] K. Erol, J. Hendler, and D. S. Nau, "HTN planning: Complexity and expressivity," in *Proc. AAAI*, pp. 1123-1128, 1994.

[17] R. E. Fikes and N. J. Nilsson, "STRIPS: A new approach to the application of theorem proving to problem solving," *Artificial Intelligence*, vol. 2, no. 3-4, pp. 189-208, 1971.

[18] Gemini Team, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[19] OpenAI, "GPT-4o," May 2024. [Online]. Available: https://openai.com/index/hello-gpt-4o/

[20] K. Greshake *et al.*, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. AISec*, pp. 79-90, 2023.

[21] D. Guo *et al.*, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[22] I. Gur *et al.*, "A Real-World WebAgent with Planning, Long Context Understanding, and Program Synthesis," *arXiv preprint arXiv:2307.12856*, 2023.

[23] S. Hong *et al.*, "MetaGPT: Merging large language model multi-agent collaboration," in *Proc. ICLR*, 2024.

[24] J. Huang and K. C.-C. Chang, "Towards reasoning in large language models: A survey," in *Findings of ACL 2023*, pp. 1049-1065, 2023.

[25] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" *arXiv preprint arXiv:2310.06770*, 2023.

[26] S. Kambhampati *et al.*, "LLMs can't plan, but can help planning in LLM-modulo frameworks," *arXiv preprint arXiv:2402.01817*, 2024.

[27] J. Y. Koh *et al.*, "VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks," in *Proc. ACL*, vol. 1, pp. 881-905, 2024.

[28] LangChain, "LangGraph: Agent orchestration framework," 2024. [Online]. Available: https://www.langchain.com/langgraph

[29] X. Li *et al.*, "A survey on LLM-based multi-agent systems," *Vicinagearth*, vol. 1, no. 1, art. 9, 2024.

[30] X. Liu *et al.*, "AgentBench: Evaluating LLMs as agents," *arXiv preprint arXiv:2308.03688*, 2023.

[31] A. Liusie, P. Manakul, and M. J. F. Gales, "LLM comparative assessment: Zero-shot NLG evaluation through pairwise comparisons using large language models," in *Proc. 18th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 139-151, 2024.

[32] A. Madaan *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46534-46594, 2023.

[33] Anthropic, "Introducing the Model Context Protocol," Anthropic News, Nov. 25, 2024. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[34] Anthropic, "MCP Governance and Community," 2024. [Online]. Available: https://github.com/modelcontextprotocol

[35] G. Mialon *et al.*, "Augmented language models: A survey," *arXiv preprint arXiv:2302.07842*, 2023.

[36] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, "GAIA: A benchmark for general AI assistants," in *Proc. ICLR*, 2024.

[37] OpenAI, "Learning to Reason with LLMs," OpenAI Research Blog, Sep. 2024. [Online]. Available: https://openai.com/index/learning-to-reason-with-llms/

[38] OpenAI, "OpenAI o1 system card," *arXiv preprint arXiv:2412.16720*, 2024.

[39] OpenAI, "Introducing SWE-bench Verified," OpenAI Research Blog, Aug. 2024. [Online]. Available: https://openai.com/index/introducing-swe-bench-verified/

[40] OpenAI, "Agents SDK documentation," OpenAI API Documentation, 2025. [Online]. Available: https://developers.openai.com/api/docs/guides/agents

[41] C. Packer *et al.*, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

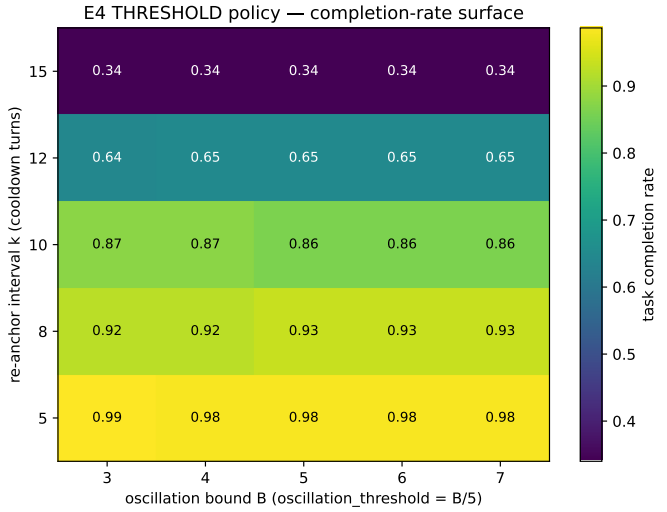


Fig. 10. E4 completion-rate surface over re-anchor cadence $k$ and oscillation bound $B$. Completion is dominated by $k$ and nearly flat in $B$. Source: simulation (E4 controller sweep).

TABLE XXVI
MCP-I02 PROMPT-INJECTION SCENARIO (OFFLINE PROOF-OF-CONCEPT, 800 TRIALS PER CONDITION; WILSON 95% CIS). SOURCE: OFFLINE PROOF-OF-CONCEPT (NO EXTERNAL HOSTS).

| Condition | Attack success | Wilson 95% CI |
|---|---|---|
| Baseline (raw tool output) | 86.3% | [83.7, 88.5] |
| Mitigated (boundary sanitizer) | 10.2% | [8.3, 12.5] |


[42] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," *arXiv preprint arXiv:2404.13076*, 2024.

[43] J. S. Park *et al.*, "Generative agents: Interactive simulacra of human behavior," in *Proc. UIST*, pp. 1-22, 2023.

[44] E. Perez *et al.*, "Discovering language model behaviors with model-written evaluations," in *Findings of ACL 2023*, pp. 13387-13434, 2023.

[45] O. Press *et al.*, "Measuring and Narrowing the Compositionality Gap in Language Models," in *Findings of EMNLP 2023*, pp. 5687-5711, 2023.

[46] S. Qiao *et al.*, "Reasoning with language model prompting: A survey," in *Proc. 61st ACL*, vol. 1, pp. 5368-5393, 2023.

[47] Y. Qin *et al.*, "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs," in *Proc. ICLR*, 2024.

[48] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in *Proc. ICMAS*, pp. 312-319, 1995.

[49] Y. Gan *et al.*, "Navigating the risks: A survey of security, privacy, and ethics threats in LLM-based agents," *arXiv preprint arXiv:2411.09523*, 2024.

[50] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[51] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems*, vol. 36, pp. 68539-68551, 2023.

[52] Y. Shen *et al.*, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," *Advances in Neural Information Processing Systems*, vol. 36, pp. 38154-38180, 2023.

[53] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 8634-8652, 2023.

[54] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM test-time compute optimally can be more effective than scaling model parameters," *arXiv preprint arXiv:2408.03314*, 2024.

[55] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents," *Transactions on Machine Learning Research*, 2023.

[56] L. Sun *et al.*, "TrustLLM: Trustworthiness in large language models," in *Proc. ICML*, 2024.

[57] Y. Talebirad and A. Nadiri, "Multi-agent collaboration: Harnessing the power of intelligent LLM agents," *arXiv preprint arXiv:2306.03314*, 2023.

[58] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think," *Advances in Neural Information Processing Systems*, vol. 36, pp. 74952-74965, 2023.

[59] K. Valmeekam, M. Marquez, S. Sreedharan, and S. Kambhampati, "On the planning abilities of large language models: A critical investigation," *Advances in Neural Information Processing Systems*, vol. 36, pp. 75993-76005, 2023.

[60] L. Wang *et al.*, "Plan-and-solve prompting," *arXiv preprint arXiv:2305.04091*, 2023.

[61] P. Wang *et al.*, "Large language models are not fair evaluators," in *Proc. 62nd ACL*, vol. 1, pp. 9440-9450, 2024.

[62] X. Wang *et al.*, "OpenDevin: An open platform for AI software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024.

[63] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824-24837, 2022.

[64] M. Wooldridge, *An Introduction to Multiagent Systems*, 2nd ed. Wiley, 2009.

[65] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversations," in *Proc. First Conference on Language Modeling (COLM)*, 2024.

[66] S. Dong, S. Xu, P. He, Y. Li, J. Tang, T. Liu, H. Liu, and Z. Xiang, "A practical memory injection attack against LLM agents," *arXiv preprint arXiv:2503.03704*, 2025.

[67] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," *arXiv preprint arXiv:2407.01489*, 2024.

[68] T. Xie *et al.*, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[69] F. Xu *et al.*, "TheAgentCompany: Benchmarking LLM agents on consequential real-world tasks," *arXiv preprint arXiv:2412.14161*, 2024.

[70] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[71] S. Yao *et al.*, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," in *Advances in Neural Information Processing Systems*, vol. 36, pp. 11809-11822, 2023.

[72] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "$\tau$-bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.

[73] J. Yi *et al.*, "Benchmarking and defending against indirect prompt injection attacks on large language models," *arXiv preprint arXiv:2312.14197*, 2023.

[74] A. Zeng *et al.*, "AgentTuning: Enabling Generalized Agent Abilities for LLMs," in *Findings of ACL 2024*, pp. 3053-3077, 2024.

[75] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proc. ISSTA*, pp. 1592-1604, 2024.

[76] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46595-46623, 2023.

[77] S. Zhou *et al.*, "WebArena: A realistic web environment for building autonomous agents," *arXiv preprint arXiv:2307.13854*, 2023.